

The MLOps Practitioner Mini Projects & Final Project Handbook

Five mini projects and one final project, in one repository. Every module ends with working code, a pull request, and a peer review from your study group. The final project is what earns your certificate.

5 MODULES

5 MINI PROJECTS

1 GITHUB REPOSITORY

1 PULL REQUEST PER MODULE

FINAL PROJECT → CERTIFICATE

PREREQUISITES

RESOURCES

OUTPUT – WHAT YOU SHIP

DEFINITION OF DONE

FAILURE MODES

PULL REQUEST & REVIEW

Every coloured block in this handbook means the same thing everywhere it appears. Learn the six colours once and you can navigate any module by scanning.

How this handbook works

The five mini projects are not five assignments. They are five layers of one system. You start with a Jupyter notebook in Week 1 and finish in Week 6 with a monitored, optimized, automatically-retraining ML service.

BLOCK	WHAT IT MEANS
Goal	One sentence — what exists at the end that did not exist at the start
Session coverage map	Which lesson maps to which task, so nothing taught is left unpractised
Prerequisites	What you must have installed, know, or have finished before starting
Steps	The exact sequence of work, with commands and an acceptance check per step
Pull request	What the PR must contain and what your reviewer is asked to check
Output	The concrete artifacts you hold at the end
Definition of done	A checklist. Any unticked box means the mini project is not finished

BLOCK	WHAT IT MEANS
Failure modes	The mistakes that cost previous students the most hours
Resources	External material to fill gaps a 3-hour session cannot cover

Effort and sequence

Each mini project is designed for **6–12 hours** of focused work in the week between sessions. They are cumulative: skipping Mini Project 1 makes Mini Project 3 impossible.

REFERENCE MODEL

All mini projects use a **ride-duration regression model** (NYC TLC trip data — small, clean, trains in under two minutes on a laptop). This is deliberate: the point is the *engineering around the model*, not the model. Do not pick something exotic that will eat your week in data cleaning.

Your **final project** is the same infrastructure with your own model swapped in. Every hour on a mini project is an hour you do not spend on the final project.

NOTE ON MODULE ORDER

This handbook follows the delivery order on the course platform: **Week 5 is Optimization** and **Week 6 is Observability**. Some earlier materials number these the other way round. The mini project content is unchanged either way — only the week it is due shifts.

READ FIRST

Collaboration protocol

MODULE 0

Environment setup

MINI PROJECT 1

Notebook → production service

MINI PROJECT 2

The pipeline that retrains itself

MINI PROJECT 3

Serve it three ways

MINI PROJECT 4

The optimization journey

MINI PROJECT 5

The observable ML system

CERTIFICATE

The final project

APPENDIX

Master resource index

The collaboration protocol

Read this before Session 1. It is not optional and it is not a suggestion — it is how the course is graded and how you will actually work in a team.

RULE 1 – ONE MODULE, ONE PULL REQUEST

Each mini project is delivered as **exactly one pull request**. You never commit to `main` directly.

```
git switch -c module-1-packaging # branch named for the module
# ... work, committing in small logical chunks ...
git push -u origin module-1-packaging
# open a Pull Request on GitHub → request review → merge after review
git tag -a v0.1.0 -m "Module 1: containerized, tested ML API"
git push --tags
```

Five modules → five branches → five pull requests → five tagged releases: `v0.1.0` through `v0.5.0`. Your commit history is your evidence of learning, and the reviewer will check out these tags.

RULE 2 – SHARE YOUR REPOSITORY WITH YOUR STUDY GROUP

In the first week, post your repository URL in the course study group. Everyone works on the same five mini projects, so everyone can read everyone else's code — that is the point.

- Post the link once, at the top of the group, and keep the repo **public**.
- Every time you open a PR, drop the PR link in the group and **ask two people to review it**.
- Review at least **two** other people's pull requests per module. Reviewing is where most of the learning happens — you see five other approaches to the same problem.
- If nobody claims your PR within 48 hours, tag the instructor.

This is deliberate practice for the mandatory **peer review** in the final project. Students who review from Week 1 write far better final reviews.

RULE 3 – WHAT A GOOD PULL REQUEST LOOKS LIKE

A PR is a piece of communication, not a code dump. Yours must have:

- **A title that names the module and the change** —
`Module 2: MLflow tracking, DVC pipeline, CI with quality gate`, not `updates`.
- **A description** with: what changed, how to run it, the numbers you measured, and one open question for your reviewer.

- **Screenshots or output** pasted in — the MLflow comparison, the green CI run, the Grafana panel. Reviewers should not have to run your code to see that it works.
- **A checked-off Definition of Done** copied from this handbook into the PR description.
- **Small, readable commits.** Commit messages describe *why*, not *what*. `fix bug` is not a commit message.

From Module 2 onward, CI runs on every PR. **A red check means the PR is not ready for review.** Fix it before asking anyone to look.

RULE 4 – HOW TO REVIEW SOMEONE ELSE'S PULL REQUEST

Leave comments on the diff itself, not just a summary. Cover four things:

1. **Can I run it?** Clone the branch, follow the README, and say exactly where it broke or that it worked first try.
2. **One thing done well**, named specifically. "Good code" is not useful feedback.
3. **Two concrete improvements**, specific enough to act on immediately. "Add more tests" is not actionable; "the API test mocks nothing, so it fails without a trained model on disk" is.
4. **One question** about a decision you did not understand. Questions teach more than corrections.

Be direct and be kind. You are reviewing the code, never the person.

WHY THIS MATTERS BEYOND THE GRADE

Nobody in production merges their own work unreviewed. The PR habit, the review habit, and a repo a stranger can run are the three things that separate a portfolio project from a production engineer's repository — and they are what a hiring manager actually looks at.



BEFORE SESSION 1 ENDS · NOT PART OF ANY SINGLE SESSION

Environment setup

Accounts, WSL2, Docker, and the repository everything else lives in. Students who postpone this lose the entire first week debugging Docker instead of writing code.

Everything you build for the next six weeks lives in **one GitHub repository** and produces **Docker images pushed to one Docker Hub account**. You will not create a new repo per session. You will not send `.zip` files. The repository is your submission, your portfolio, and your grade.

0.1 — Create your accounts

ACCOUNT	WHERE	WHY YOU NEED IT
GitHub	github.com/signup	Code, pull requests, CI/CD workflows, releases, and your final submission
Docker Hub	hub.docker.com/signup	Publishing your container images so anyone can <code>docker run</code> your model
Weights & Biases (optional)	wandb.ai	Alternative to MLflow in Module 2

1. Create the GitHub account with an email you will still have after graduation. Add a photo and a real name — recruiters read this.
2. Create the Docker Hub account. **Use the same username as GitHub if it is available** — it makes every command shorter and your image names cleaner.
3. On Docker Hub: **Account Settings → Personal access tokens → Generate new token** with *Read, Write, Delete* scope. Copy it into a password manager immediately — it is shown once. GitHub Actions needs it in Module 2.
4. Add an SSH key to GitHub so you stop typing passwords:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
cat ~/.ssh/id_ed25519.pub      # paste into GitHub → Settings → SSH and GPG keys
ssh -T git@github.com         # should greet you by username
```

0.2 — Install WSL2 (Windows only)

If you are on Windows, **you are not going to develop on Windows**. You are going to develop on Linux running inside Windows. Every production ML system you will ever touch runs on Linux, every tutorial assumes Linux paths, and half the tooling in this course either breaks or behaves differently on native Windows.

Open **PowerShell as Administrator**:

```
wsl --install -d Ubuntu-22.04
```

1. Restart when prompted.
2. Open the **Ubuntu** app. Create your Linux username and password (separate from your Windows login — write it down).
3. Verify you are on version 2: `wsl -l -v` → the `VERSION` column must read `2`. If it reads `1`, run `wsl --set-version Ubuntu-22.04 2`.
4. Update the system inside Ubuntu:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential curl git python3-pip python3-venv
```

CRITICAL – THE MISTAKE THAT COSTS A WEEK

Keep all project files **inside the Linux filesystem** (`/home/<you>/projects/...`), never in `/mnt/c/Users/...`. Cross-filesystem I/O in WSL is 10–20× slower and will make Docker builds and test runs feel broken when they are only slow.

Recommended: install VS Code on Windows plus the **WSL** extension, then run `code` from inside your Ubuntu terminal.

macOS / Linux users: skip this section. You are already there.

0.3 — Install Docker

Docker is the unit of deployment for everything after Week 1. It is not a convenience — it is how your model reaches production.

OS	WHAT TO INSTALL
Windows + WSL2	Docker Desktop for Windows, then <i>Settings</i> → <i>Resources</i> → <i>WSL Integration</i> and enable your Ubuntu distro
macOS	Docker Desktop for Mac — pick the Apple Silicon or Intel build that matches your chip
Linux	Docker Engine, then <code>sudo usermod -aG docker \$USER && newgrp docker</code> so you do not need <code>sudo</code>

Verify — all four must succeed:

```
docker --version
docker compose version
docker run --rm hello-world
docker login -u <your-dockerhub-username>
```

ACCEPTANCE CHECK

If `docker run --rm hello-world` prints a greeting, your installation is correct. If it hangs or errors, fix it now — every remaining week depends on it.

0.4 — Create the course repository

Create the repo on GitHub first — **public**, initialized with a README and a Python `.gitignore`. Then:

```
mkdir -p ~/projects && cd ~/projects
git clone git@github.com:<your-username>/mlops-practitioner.git
cd mlops-practitioner
```

The repository contract

This is the structure your repo grows into by Week 6. Create directories as you reach them — do not create empty folders in Week 1.

```
mlops-practitioner/
├── README.md                # the 3-command quickstart – updated every module
├── pyproject.toml           # M1
├── .gitignore               # M1
├── .pre-commit-config.yaml  # M1
├──
├── src/prodml/              # M1 – the installable package
│   ├── config.py            logging_conf.py
│   ├── data.py              features.py  train.py  export.py
│   └── api/ (main.py, schemas.py)
├──
├── tests/                   # M1 – grows every module
├── docker/                  # M1 – Dockerfile, docker-compose.yml
├──
├── pipelines/               # M2 – dvc.yaml; M3 – Airflow DAGs
├── infra/                   # M2 – Terraform
├── .github/workflows/       # M2 – CI, continuous training
├──
├── serving/                 # M3 – BentoML, Triton, vLLM, nginx configs
├── loadtest/                # M3 – Locust files + results
├── optimization/           # M4 – benchmark harness + one script per technique
├── monitoring/              # M5 – Evidently, Prometheus, Grafana, Langfuse
├──
├── reports/                 # your lab report per module – this is graded
│   └── module-1.md ... module-5.md
└── docs/
```

NEVER COMMIT

`.env` • credentials and API keys • `*.pkl` over 50 MB • raw files in `data/` (DVC handles those from Module 2) • `__pycache__` • `.venv` • `*.tfstate`

NO MAKEFILE IN THIS COURSE

Type the real commands. A `Makefile` hides what is actually running, and when a step fails inside a target you will not know which one. Put the commands your project needs in the **README** instead — that is what your reviewer reads anyway, and the README is graded.

01

MODULE 1 • FROM NOTEBOOK TO PRODUCTION-READY CODE • 7 LESSONS

Mini Project 1 — From notebook to production-ready service

Goal: take a working `.ipynb` and turn it into a pip-installable package, exposed as a tested FastAPI service, running in a multi-stage Docker container, published to Docker Hub, that any engineer can run in three commands.

Session coverage map

LESSON	WHERE IT APPEARS
1 — MLOps maturity model	Step 8 — self-assess your repo against the five levels
2 — Python packaging & project structure	Step 2 — <code>pyproject.toml</code> , <code>src/</code> layout, OOP, type hints, decorators
3 — Building ML APIs (FastAPI / Litestar)	Step 5 — <code>/predict</code> , <code>/health</code> , <code>/metadata</code> with Pydantic
4 — Serialization formats	Step 4 — pickle vs ONNX, parity check, format decision
5 — Docker & containerization	Step 7 — multi-stage build, compose, Docker Hub push
6 — Structured logging	Step 3 — JSON logs, correlation IDs, log levels
7 — Testing ML code with pytest	Step 6 — fixtures, mocks, coverage gate

PREREQUISITES

Completed: all of Module 0 — repo cloned, Docker Hub token saved, `docker run hello-world` passing, WSL2 on version 2 if on Windows.

Knowledge you must already have:

- Python fundamentals: functions, classes, imports, virtual environments
- Having trained *at least one* model before (any Kaggle notebook counts). If you have never trained one, complete **Course 1 of Andrew Ng's ML Specialization** before Week 2 or this course will feel abstract
- Basic terminal usage: `cd`, `ls`, `mkdir`, `cat`, pipes

Machine: any laptop, no GPU, ~10 GB free disk for Docker images. **Time:** 6–8 hours.

Steps

01 Establish the baseline 45 min

1. Create `notebooks/00-baseline.ipynb`. Download one month of NYC TLC green taxi trip data (Parquet).
2. Build the minimum viable model: engineer `PU_D0` (pickup-dropoff pair) and `trip_distance`, use `DictVectorizer` + `LinearRegression` or `RandomForestRegressor`, target is `duration` in minutes.
3. Print validation **RMSE and MAE**. Write both at the top of `reports/module-1.md`.
4. Save the fitted model to `models/baseline.pkl`.

This notebook is deliberately messy. It is the "before" picture — **commit it as-is** and point at it in your report to show what you replaced.

ACCEPTANCE CHECK

The notebook runs top to bottom with *Restart & Run All* and prints a validation MAE.

02 Turn it into a real Python package 90 min

1. Create the layout and environment:

```
mkdir -p src/prodml/api tests
python3 -m venv .venv && source .venv/bin/activate
```

2. Write `pyproject.toml` with metadata, dependencies, and a script entry point:

```
[project]
name = "prodml"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["scikit-learn", "pandas", "pyarrow", "fastapi", "uvicorn",
               "pydantic", "pydantic-settings"]

[project.optional-dependencies]
dev = ["pytest", "pytest-cov", "ruff", "black", "mypy", "httpx", "pre-commit"]

[project.scripts]
prodml-train = "prodml.train:main"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
```

3. Install in editable mode — the moment your code becomes a *package* rather than a folder of scripts:

```
pip install -e ".[dev]"
python -c "import prodml; print(prodml.__file__)"
```

4. Decompose the notebook. Every cell must land in exactly one place:

NOTEBOOK RESPONSIBILITY	MODULE
loading Parquet, train/val split	<code>src/prodml/data.py</code>
feature engineering	<code>src/prodml/features.py</code>
model fit + persist	<code>src/prodml/train.py</code>
load model + single/batch predict	<code>src/prodml/predict.py</code>
all paths, hyperparameters, ports	<code>src/prodml/config.py</code>

5. Apply the session's engineering practices:

- **Type hints on every public function** — `def predict(self, features: dict[str, Any]) -> float:`
- **One class with a clear interface**, e.g. `DurationPredictor` with `.load()`, `.predict_one()`, `.predict_batch()`. This is the seam that makes Modules 3, 4 and 5 possible — the API, the BentoML runner, and the optimized ONNX model all sit behind it
- **A decorator you wrote yourself**, e.g. `@timed` logging execution time, applied to `predict_one`. You will thank yourself in Module 4
- **Config via `pydantic-settings`**, read from environment variables with sane defaults. **No hardcoded paths anywhere**

6. Add `.pre-commit-config.yaml` with `ruff`, `black` and `end-of-file-fixer`, then `pre-commit install`.

7. Put the commands your project needs into the README — these five are the whole workflow:

```
pip install -e ".[dev]"           # install
ruff check src tests && black --check src tests  # lint
pytest -v --cov=src/prodml --cov-report=term-missing # test
python -m prodml.train              # train
uvicorn prodml.api.main:app --reload --port 8000    # serve
```

ACCEPTANCE CHECK

`python -m prodml.train` produces `models/model.pkl` with the same MAE (± 0.05) as your notebook. If the number moved more than that, you changed logic while refactoring — find it.

03 Structured logging 30 min

1. In `src/prodml/logging_conf.py`, configure a JSON formatter (`python-json-logger` or a small custom `logging.Formatter`).

2. Every log line carries: `timestamp`, `level`, `logger`, `message`, `correlation_id`.
3. Generate a **correlation ID** (`uuid4`) in FastAPI middleware, attach it via `contextvars`, return it as an `X-Request-ID` header.
4. Use levels correctly and prove you understand them — `DEBUG`: feature vector • `INFO`: prediction served with latency • `WARNING`: input outside the training range (`trip_distance > 100`) • `ERROR`: model load failure, validation rejection.
5. Delete every `print()` from `src/`.

ACCEPTANCE CHECK

`curl` your API and pipe logs through `jq` — valid JSON, and the same `correlation_id` appears in the request log, the prediction log, and the response header.

04 Serialization: choose your format deliberately 45 min

1. In `src/prodml/export.py`, export to **ONNX** (`skl2onnx` for scikit-learn, `torch.onnx.export` for PyTorch) with dynamic axes on the batch dimension.
2. Write a **parity test**: run 500 validation rows through both the pickle model and the ONNX model via `onnxruntime`, assert `np.allclose(pred_pkl, pred_onnx, atol=1e-4)`.
3. Benchmark both on the same 500 rows: mean latency and p95, pickle vs ONNX, into your report.
4. Write a comparison table covering **JSON, Protobuf, Pickle, ONNX** on four axes: human-readable, cross-language, schema-enforced, safe to load from an untrusted source. Then state in one sentence which format your service serves with and why.

THE THING TO REMEMBER

Pickle executes arbitrary code on load. Never load a `.pkl` you did not produce. Say this explicitly in your report.

ACCEPTANCE CHECK

Parity test passes; both latency numbers are in your report.

05 Build the API 75 min

1. In `src/prodml/api/schemas.py` define Pydantic models: `PredictionRequest` with `Field(...)` constraints (`trip_distance: float = Field(gt=0, lt=200)`) and a `model_config` example for the docs; `PredictionResponse` with `prediction`, `model_version`, `correlation_id`, `latency_ms`; plus batch variants.
2. In `src/prodml/api/main.py` implement four endpoints:

ENDPOINT	METHOD	BEHAVIOUR
<code>/health</code>	GET	<code>200</code> only if the model object is loaded in memory — not just "the process is alive"
<code>/metadata</code>	GET	model version, training date, feature names, framework, artifact hash
<code>/predict</code>	POST	single prediction
<code>/predict/batch</code>	POST	list in, list out

3. **Load the model once at startup** using FastAPI's `lifespan` context manager — never inside the request handler. Loading per-request is the most common beginner mistake and it costs 100× in latency.
4. Add an exception handler turning validation errors into a clean `422` with a useful message, and unexpected errors into a `500` that logs the traceback but does **not** leak it to the client.
5. Run the server, open `localhost:8000/docs`, and make a real prediction from the Swagger UI.
6. *Optional*: implement the same `/predict` in **Litestar** on a side branch and note the ergonomic difference in your report.

ACCEPTANCE CHECK

All four endpoints respond correctly; `/docs` renders your examples; sending `trip_distance: -5` returns `422` with a readable message rather than a stack trace.

06 Test suite with a coverage gate 60 min

FILE	WHAT IT MUST COVER
<code>conftest.py</code>	fixtures: <code>sample_features</code> , <code>trained_model</code> (session-scoped), <code>client</code> (FastAPI <code>TestClient</code>)
<code>test_features.py</code>	feature engineering, including edge cases: missing category, zero distance, unseen <code>PU_D0</code> pair
<code>test_predict.py</code>	prediction returns a float, in a sane range, deterministic across two calls
<code>test_api.py</code>	<code>/health</code> returns 200; <code>/predict</code> happy path; invalid payload returns 422; response schema matches
<code>test_serialization.py</code>	the pickle ↔ ONNX parity assertion from Step 4

- Use `monkeypatch` or `unittest.mock` in at least one test so the API test does not depend on a real training run.
- Use `@pytest.mark.parametrize` for the feature edge cases.
- Enforce the gate in `pyproject.toml`:

```
[tool.pytest.ini_options]
addopts = "--cov=src/prodml --cov-fail-under=70"
```

ACCEPTANCE CHECK

Tests pass with coverage $\geq 70\%$. Then deliberately break a feature function and confirm a test goes red — a suite that never fails is not a test suite.

07 Containerize and publish 75 min

1. Write `docker/Dockerfile` as a **multi-stage build**:

```
FROM python:3.11-slim AS builder
WORKDIR /build
COPY pyproject.toml ./
COPY src/ ./src/
RUN pip install --no-cache-dir --prefix=/install .

FROM python:3.11-slim AS runtime
RUN useradd --create-home --uid 1000 appuser
COPY --from=builder /install /usr/local
COPY --chown=appuser:appuser models/ /app/models/
WORKDIR /app
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request;urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "prodml.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2. Write `.dockerignore` (`.git`, `.venv`, `notebooks/`, `data/`, `tests/`, `__pycache__`, `*.ipynb`). Compare image sizes with and without it — both numbers go in your report.
3. **Record single-stage vs multi-stage image size.** The gap is the whole lesson.
4. Write `docker/docker-compose.yml` with the API service, environment variables, a volume mount for `models/`, port mapping, and `restart: unless-stopped`.
5. Build, tag with a semantic version **and** `latest`, and push:

```
docker build -f docker/Dockerfile -t <user>/prodml-api:0.1.0 -t <user>/prodml-api:latest .
docker run --rm -p 8000:8000 <user>/prodml-api:0.1.0
docker push <user>/prodml-api:0.1.0 && docker push <user>/prodml-api:latest
```

6. Confirm non-root: `docker exec <id> whoami` → `appuser`.

ACCEPTANCE CHECK

A classmate can run `docker run -p 8000:8000 <your-user>/prodml-api:0.1.0` on their machine, having cloned nothing, and get a prediction.

08 README, maturity self-assessment, release 30 min

1. Rewrite `README.md` so a stranger goes from zero to a prediction in **three commands**. Include: a one-paragraph description, the three commands, an example `curl` with real JSON, and the repo tree.
2. In `reports/module-1.md`, add a **maturity self-assessment**: place your repo on the five-level model from Lesson 1 and state in two sentences what is missing to reach the next level. (Spoiler: it is Module 2.)

PULL REQUEST – MODULE 1

Branch: `module-1-packaging` → **Tag after merge:** `v0.1.0`

Paste into the PR description: your before/after MAE, the pickle-vs-ONNX latency numbers, the single-stage vs multi-stage image size, your Docker Hub image URL, and the ticked Definition of Done.

Ask your reviewers to check specifically: can they pull your image and get a prediction without cloning? Does the model load at startup rather than per request? Are there any `print()` statements left in `src/` ?

OUTPUT – WHAT YOU SHIP

1. A **pip-installable Python package** with a clean boundary between data, features, training and prediction
2. A **FastAPI service** with 4 endpoints, Pydantic-validated I/O, and startup model loading
3. **Structured JSON logs** with correlation IDs traceable end to end
4. A **model in two serialization formats** with a passing parity test and measured latency comparison
5. A **pytest suite** with fixtures, mocks, parametrization and an enforced 70% coverage gate
6. A **multi-stage Docker image** running as non-root, published publicly on Docker Hub
7. A **3-command README**, a merged pull request, and a tagged `v0.1.0` release
8. A written **maturity self-assessment**

DEFINITION OF DONE

- ☐ GitHub repo public with more than one commit; Docker Hub token saved securely
- ☐ `docker run hello-world` works; WSL2 on version 2 (Windows)
- ☐ `pip install -e .` succeeds in a clean virtualenv
- ☐ Lint passes; `pre-commit` hooks installed
- ☐ Tests pass with coverage $\geq 70\%$

- ☐ Zero `print()` statements in `src/`
- ☐ `/health`, `/metadata`, `/predict`, `/predict/batch` all respond correctly
- ☐ ONNX parity test passes
- ☐ Image pushed to Docker Hub and pullable by someone else; container runs as non-root
- ☐ `README.md` gets a stranger to a prediction in 3 commands
- ☐ `reports/module-1.md` has: MAE, latency comparison, image-size comparison, serialization table, maturity self-assessment
- ☐ Pull request opened, reviewed by two peers, merged; `v0.1.0` tagged

FAILURE MODES

MISTAKE	COST
Loading the model inside the request handler	100× latency; you will "discover" it painfully in Module 3's load test
Working in <code>/mnt/c/...</code> under WSL	Docker builds that take 8 minutes instead of 40 seconds
Committing <code>data/</code> and <code>.venv</code>	A repo that takes minutes to clone, and a rejected PR
Copying <code>.venv</code> into the image	A 3 GB image
Writing tests only after the code	Tests that assert whatever the code already does, catching nothing
Hardcoded absolute paths	Works on your laptop, fails in the container. Every time

RESOURCES – MODULE 1

PROVIDED BY THE INSTRUCTOR

- Git — full course: youtube.com/watch?v=N7qWNQxFq90
- Linux — full course: youtube.com/watch?v=6WatcfENsOU
- Docker session (MLOps MENA on Zomra): zomra.io free session

ENVIRONMENT SETUP

- Install WSL2 — official Microsoft guide: learn.microsoft.com/windows/wsl/install
- Docker Desktop: docs.docker.com/desktop • Get Started: docs.docker.com/get-started
- GitHub SSH keys: docs.github.com — [connecting with SSH](#)
- GitHub pull requests — how to open and review one: docs.github.com/pull-requests

PACKAGING & CODE QUALITY

- Python Packaging User Guide: packaging.python.org
- Ruff (linter + formatter): docs.astral.sh/ruff • pre-commit: pre-commit.com
- `uv` — a much faster pip/venv replacement worth adopting: docs.astral.sh/uv

APIS, TESTING, SERIALIZATION

- FastAPI — read the whole "Tutorial — User Guide": fastapi.tiangolo.com/tutorial
- Pydantic v2: docs.pydantic.dev • Litestar: docs.litestar.dev
- pytest: docs.pytest.org • fixtures deep dive: docs.pytest.org — fixtures
- ONNX Runtime: onnxruntime.ai/docs • `skl2onnx` : onnx.ai/sklearn-onnx • Protobuf: protobuf.dev

BACKGROUND READING

- *Made With ML* — free, project-based, strongest free material on packaging and testing: madewithml.com
- Google Cloud, *MLOps: Continuous delivery and automation pipelines in ML* — the source of the maturity-level framing: cloud.google.com/architecture

02

MODULE 2 · MLOPS CORE – TRACKING, VERSIONING, AUTOMATION · 6 LESSONS

Mini Project 2 — The pipeline that retrains itself

Goal: make every model reproducible and auditable, then remove yourself from the retraining loop — a pipeline that pulls data, trains, evaluates against the current production model, promotes only if better, and builds a deployable image, with zero clicks.

Session coverage map

LESSON	WHERE IT APPEARS
1 — MLflow experiment tracking	Step 2 — params/metrics/artifacts for 3 model families
2 — Model registry	Step 3 — None → Staging → Production; load by stage, never by path
3 — Continuous training	Step 7 — triggers, the gate, promotion logic
4 — Data versioning with DVC	Step 4 — remote storage, <code>dvc repro</code> , DVC ↔ MLflow lineage
5 — CI/CD with GitHub Actions	Step 5 — lint → test → gate → build → push
6 — Infrastructure as code with Terraform	Step 6 — provision the artifact store reproducibly

PREREQUISITES

Completed: Mini Project 1, merged and tagged `v0.1.0`. You need a working `prodml.train` entry point and a `DurationPredictor` class — Module 2 wires machinery *around* them.

Knowledge: YAML syntax; environment variables and secrets; what a cloud object store is (S3 / GCS / Azure Blob).

Access: GitHub Actions is enabled by default on public repos. For DVC remote storage pick **one**: Google Drive (free, easiest), an AWS S3 bucket (free tier), or self-hosted **MinIO** via Docker Compose — recommended if you have no card.

Machine: no GPU, ~15 GB disk (MLflow artifacts and the DVC cache grow fast). **Time:** 8–10 hours — the heaviest mini project.

Steps

01 Run the tracking infrastructure locally 45 min

1. Extend `docker/docker-compose.yml` with an **MLflow tracking server**, a **PostgreSQL** backend store, and **MinIO** as the S3-compatible artifact store.
2. Confirm the MLflow UI at `:5000` and the MinIO console at `:9001`.
3. Set `MLFLOW_TRACKING_URI` in `.env` and read it through `prodml.config`.

Running MLflow with a *file* backend is the fast path, but you learn nothing about how it is deployed. The Postgres + object-store topology here is exactly what you will meet at work.

ACCEPTANCE CHECK

`mlflow.set_tracking_uri(...)` from a Python shell lists your experiments without error.

02 Track three model families properly 90 min

1. Refactor `prodml/train.py` so training is wrapped in `with mlflow.start_run(run_name=...)`.
2. Train and log **three model families** on identical splits: Linear Regression (always have a baseline), XGBoost, and a small PyTorch MLP.
3. For **every** run log:
 - **Params** — all hyperparameters, the split seed, the data version hash
 - **Metrics** — RMSE, MAE, R^2 , train duration in seconds, model size in MB
 - **Artifacts** — the model, a residual plot, a feature-importance plot, the exact `requirements.txt`
 - **Tags** — `git_commit`, `data_version`, `author`, `framework`
4. Turn on **autologging** for one framework (`mlflow.xgboost.autolog()`) and note what it captured for free versus what you had to log manually.
5. Run a **hyperparameter sweep** on XGBoost — at least 10 runs (`optuna`, `hyperopt`, or a plain loop). Each trial is a nested run.
6. Sort the UI by MAE and screenshot the comparison view for `reports/module-2.md`.

ACCEPTANCE CHECK

The UI shows ≥ 13 runs, and from the UI alone you can answer: which hyperparameters produced the best MAE, on which data version, from which git commit?

03 Model registry and the promotion lifecycle 60 min

1. Register your best run's model as `ride-duration-predictor`.
2. Walk it through the lifecycle deliberately: version 1 → **Staging** → **Production**. Register a worse model as version 2 and leave it in **None** for contrast.
3. Rewrite `prodml/predict.py` to load by stage, not by path:

```
model = mlflow.pyfunc.load_model("models:/ride-duration-predictor/Production")
```

4. Prove the point: promote version 2 to Production in the UI, restart the API container **without changing a single line of code**, and confirm the predictions changed. Screenshot it — this is the single most important idea in Module 2.
5. Add `prodml/registry.py` with `promote_if_better(candidate_run_id, metric="mae")` comparing against the current Production model.

ACCEPTANCE CHECK

Switching the served model requires zero code changes and zero rebuilds.

04 Version your data with DVC 90 min

1. Initialize and configure the remote:

```
dvc init
dvc remote add -d storage s3://prodml-dvc # or gdrive:// or your MinIO endpoint
dvc add data/raw/green_tripdata_2023-01.parquet
git add data/raw/*.dvc .gitignore && git commit -m "track January data with DVC"
dvc push
```

2. **Prove versioning works** — this is the exercise, not the setup: add February's data, `dvc add`, `commit`, `dvc push`; then `git checkout` the January commit, run `dvc checkout`, and confirm `data/` reverted. Document what you observed.
3. Define a pipeline in `pipelines/dvc.yaml` with stages `prepare → featurize → train → evaluate`, each declaring `deps`, `params`, `outs`, `metrics`.
4. Run `dvc repro`, then run it again unchanged and observe the caching. Change one value in `params.yaml` and observe that only downstream stages re-run.
5. Use `dvc metrics show` and `dvc metrics diff` to compare two commits.
6. **Wire the lineage:** log the DVC data hash as an MLflow tag on every run, so any model in the registry traces back to the exact bytes it was trained on.

ACCEPTANCE CHECK

From any MLflow run ID you can recover the exact dataset via its DVC hash, and `dvc repro` reproduces the reported metric.

05 CI with GitHub Actions 90 min

Create `.github/workflows/ci.yaml`, triggered on `pull_request` and `push` to `main`:

JOB STEPS

```
lint checkout → setup-python → cache pip → ruff check → black --check
```

JOB STEPS

`test` needs `lint` → `pip install -e ".[dev]"` → `pytest --cov --cov-fail-under=70` → upload coverage artifact

`build` needs `test` → `docker/login-action` with your Docker Hub token → `buildx build` → push `sha-<short>` and `latest`

1. Store `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` in **Settings → Secrets and variables → Actions**. Never in the YAML.
2. Add a **model quality gate**: a script that loads the new run's metric and the current Production model's, and calls `sys.exit(1)` if MAE regressed by more than 5%. **A red CI check on a metric regression is the point of this whole step.**
3. Deliberately break something and screenshot the red check for your report. Then fix it and screenshot the green one.
4. Enable **branch protection** on `main` requiring CI to pass before merge — this is what makes Rule 1 of the collaboration protocol enforceable.
5. Add the CI status badge to `README.md`.

ACCEPTANCE CHECK

Opening a PR runs all three jobs; a metric regression blocks the merge.

06 Infrastructure as code with Terraform 60 min

1. In `infra/`, write Terraform that provisions the storage your pipeline depends on. Pick the path that matches your access:
 - **Cloud path**: an S3 or GCS bucket for DVC remote + MLflow artifacts, with versioning enabled and public access blocked
 - **No-cloud path**: use the Docker provider to declare your MinIO + Postgres + MLflow containers as code
2. Structure it properly: `main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`. No values hardcoded in `main.tf`.
3. Run the full loop and capture each output: `terraform init` → `fmt` → `validate` → `plan` → `apply`.
4. Then run `terraform destroy` and `terraform apply` again. **Reproducing your infrastructure from nothing, in one command, is the deliverable.**
5. Add `*.tfstate*` and `.terraform/` to `.gitignore`, and write one paragraph on why state files must never be committed and what remote state backends solve.

ACCEPTANCE CHECK

`terraform destroy` followed by `terraform apply` returns you to a working environment.

07 Close the loop: continuous training 75 min

Create `.github/workflows/continuous-training.yml` implementing the full CT pipeline:

```
trigger → dvc pull → validate data → train → evaluate
      → gate (better than Production?) → promote to Staging → build image → notify
```

1. Configure **three trigger types**: `schedule:` a weekly cron • `workflow_dispatch:` manual, with a data-version input • `repository_dispatch:` an external webhook. **The last one is the hook Module 5's drift detector fires into — leave it wired and ready.**
2. Implement the **promotion gate**: promote to Staging only if the candidate beats Production MAE by a configurable margin. Otherwise log the attempt and exit 0 — a failed challenge is not a build failure.
3. **Never auto-promote to Production.** Staging is automatic; Production requires a human approving a GitHub Environment. Defend this design choice in two sentences.
4. Trigger the workflow manually and screenshot a full green run.
5. Write the **CT decision table**: for each of the four triggers from the session — schedule, data drift, performance degradation, new labeled data — state which your pipeline implements now and which Module 5 will add.

ACCEPTANCE CHECK

A manual `workflow_dispatch` run completes end to end, producing a new Staging model version and a new image on Docker Hub.

PULL REQUEST – MODULE 2

Branch: `module-2-tracking-automation` → **Tag after merge:** `v0.2.0`

This is the first PR where **CI runs on your pull request**. Do not request review until the checks are green.

Paste into the PR: the MLflow comparison screenshot, the red-then-green CI screenshots, the `terraform plan` output, and the CT decision table. **Ask your reviewers to check:** can they `dvc pull` and reproduce your metric? Does anything still load the model by hardcoded path? Are any secrets visible in the workflow YAML?

OUTPUT – WHAT YOU SHIP

1. A running **MLflow stack** (server + Postgres + object store) via Compose
2. **13+ tracked runs** across three model families with full params, metrics, artifacts and git/data tags
3. A **model registry** driving your serving layer — swapping the production model needs no code change

4. **DVC-versioned data** with a working remote and a reproducible `dvc.yaml` pipeline
5. **Full lineage**: model version → MLflow run → git commit → DVC data hash
6. A **CI pipeline** that lints, tests, gates on model quality, and publishes images
7. **Terraform-provisioned infrastructure** destroyable and recreatable in one command
8. A **continuous training workflow** with three triggers and a human-approval gate before Production

DEFINITION OF DONE

- ☐ MLflow reachable at `:5000` with Postgres backend and object-store artifacts
- ☐ ≥ 13 runs; ≥ 3 model families; ≥ 10 -trial sweep; every run tagged with `git_commit` and `data_version`
- ☐ Model registered and moved None → Staging → Production
- ☐ `predict.py` loads by stage URI, never by file path
- ☐ Swapping Production model changes served predictions with no code change (screenshot)
- ☐ `dvc push` / `dvc pull` work against a real remote; `dvc repro` reproduces the metric; caching demonstrated
- ☐ CI runs on every PR: lint → test → gate → build → push; secrets in GitHub Secrets only
- ☐ A metric regression blocks a merge (red + green screenshots); branch protection enabled on `main`
- ☐ `terraform destroy` → `terraform apply` restores a working environment
- ☐ CT workflow has schedule + manual + `repository_dispatch` triggers; Production promotion requires human approval
- ☐ `reports/module-2.md` complete; PR reviewed by two peers and merged; `v0.2.0` tagged

FAILURE MODES

MISTAKE	COST
Logging metrics but not params, or vice versa	Runs you cannot reproduce — the entire point is lost
Different data splits between model families	An invalid comparison; your "best model" is noise
Loading the model by hardcoded path "just for now"	Modules 3 and 5 both break; you will rewrite it anyway
Committing <code>data/*.parquet</code> alongside DVC	A repo nobody can clone, and DVC doing nothing
Secrets pasted into workflow YAML	A leaked Docker Hub token on a public repo, within hours

MISTAKE**COST**

A CI pipeline with no failing condition

A green checkmark that means nothing

Auto-promoting to Production

The scenario where a silently-broken model ships on a Friday

RESOURCES – MODULE 2**EXPERIMENT TRACKING**

- MLflow docs: mlflow.org/docs · Model Registry guide: mlflow.org/docs/model-registry
- Weights & Biases quickstart: docs.wandb.ai/quickstart · Optuna: optuna.readthedocs.io

DATA VERSIONING

- DVC Get Started: dvc.org/doc/start · Pipelines: dvc.org/doc/user-guide/pipelines
- DVC experiment management: dvc.org/doc/user-guide/experiment-management

CI/CD

- GitHub Actions docs: docs.github.com/actions · Workflow syntax: [workflow syntax reference](#)
- `docker/build-push-action`: github.com/docker/build-push-action
- CML — post metric reports into your pull requests automatically: cml.dev/doc

INFRASTRUCTURE AS CODE

- Terraform tutorials — start with "Get Started — Docker" if you have no cloud account: developer.hashicorp.com/terraform/tutorials
- AWS provider: registry.terraform.io — AWS · Docker provider: registry.terraform.io — Docker

COMPANION MATERIAL

- **MLOps Zoomcamp** — free and self-paced; Modules 2, 3 and 6 map almost exactly onto this mini project, and it uses the same NYC TLC dataset: github.com/DataTalksClub/mlops-zoomcamp
- Sculley et al., *Hidden Technical Debt in Machine Learning Systems* (2015) — read once; it explains why this whole module exists: [NeurIPS 2015](#)

03

MODULE 3 · INFERENCE, SERVING AND RELEASE STRATEGIES · 11 LESSONS

Mini Project 3 — Serve it three ways, then release it safely

Goal: turn one registered model into a production serving system — orchestrated by Airflow, exposed through three inference patterns, served across multiple runtime categories, load-tested to its breaking point, and updated via a canary rollout that rolls itself back when latency degrades.

Session coverage map

LESSON	WHERE IT APPEARS
1 — Orchestration with Airflow	Step 1 — the extract → train → evaluate → register DAG
2 — Why inference patterns matter	Step 2 — the written decision, <i>before</i> any tool is chosen
3 — Three inference patterns	Steps 4, 5, 6 — web service, batch scoring, streaming
4 — What is model serving?	Step 2 — vocabulary and the 4-layer stack in your report
5 — CAT 1: FastAPI	Step 3 — baseline, and the four problems demonstrated with evidence
6 — CAT 2: BentoML	Step 4 — Runners, micro-batching, versioned artifacts
7 — CAT 3: TensorRT + Triton	Step 7a — GPU track
8 — CAT 4: ONNX Runtime + OpenVINO	Step 7b — CPU track, mandatory for everyone
9 — CAT 5: vLLM	Step 7c — LLM track, required for final project Track B
10 — Load testing with Locust	Step 8 — ramp to 100 users, p50/p95/p99, find the bottleneck
11 — Release strategies	Step 9 — nginx canary with automatic rollback, plus shadow mode

PREREQUISITES

Completed: Mini Projects 1 and 2. You need a model in the MLflow **Production** stage and an image on Docker Hub — Module 3 consumes both.

Knowledge: how HTTP requests and reverse proxies work; what a percentile is and why p95 matters more than the mean; Compose networking (service names as hostnames).

Machine — minimum: 8 GB RAM, 4 cores, ~25 GB disk. Airflow + MLflow + serving + Redis together are heavy; close your browser tabs.

GPU (optional): Steps 7a and 7c need an NVIDIA GPU. **If you have no GPU you are not blocked** — do 7b, and complete 7a in Google Colab or Kaggle's free GPU sessions as a notebook exercise. Document that you did it that way; that is an honest engineering constraint, not a failure. **Time:** 9–12 hours.

Steps

01 Orchestrate the training pipeline with Airflow 2 h

1. Add Airflow to your stack (Compose, `LocalExecutor`, Postgres metadata DB). UI at `:8080`.
2. Write `pipelines/dags/train_pipeline.py`:

```
extract → validate → train → evaluate → [branch] → register → notify
                                   └──→ skip_registration
```

3. Requirements, each mapping to a concept from the lesson:
 - **Operators** — `PythonOperator` per task; a `BranchPythonOperator` for the promotion decision
 - **XCom** — pass the MLflow `run_id` and the evaluation MAE between tasks. Pass *references*, never DataFrames; write the XCom size-limit reason in your report
 - **Sensor** — a `FileSensor` (or `S3KeySensor`) that waits for new data before the DAG proceeds
 - **Retries** — `retries=2`, `retry_delay=timedelta(minutes=5)`, `execution_timeout` on training
 - **Schedule** — weekly, `catchup=False`, `start_date` in the past
 - **Idempotency** — running the DAG twice for the same logical date must not corrupt anything
4. Trigger a **backfill** over three past dates and explain in your report what actually happened.
5. Point `register` at the same MLflow registry from Module 2. **This DAG replaces the training half of your GitHub Actions CT workflow** — write two sentences on the division of labour: why orchestration belongs in Airflow and CI belongs in GitHub Actions.

ACCEPTANCE CHECK

The DAG runs green end to end and produces a new model version in MLflow. The Graph view screenshot goes in your report.

02 Choose your patterns before you write serving code 30 min

Before any tool, write a section in `reports/module-3.md` covering:

1. **The three failure modes** — cost blowout, latency SLA breach, over-engineering — with a concrete one-line example of each for a ride-duration model.
2. **A decision table.** For three scenarios — (a) a rider app showing an ETA at booking, (b) a nightly finance report scoring 40 M historical trips, (c) surge pricing reacting to live GPS events — state which inference pattern you would choose and the SLA that drives it.

3. **The serving vocabulary** in your own words: latency vs throughput, p50/p95/p99, cold start, GPU utilization, batch size.
4. **The 4-layer serving stack** diagram, drawn for *your* system.

WHY WRITTEN FIRST

Every wrong serving architecture you will encounter in industry started with someone opening a tool's documentation before answering these four questions.

03 CAT 1 — the FastAPI baseline, and its four problems 60 min

1. Your Module 1 API is already CAT 1. Deploy it and **measure** — do not assume.
2. Run a quick Locust ramp to 50 users; record p50/p95/p99 and requests/sec.
3. Demonstrate all four CAT-1 problems **with evidence, not assertion**:

PROBLEM	HOW YOU PROVE IT
Batch size is always 1	Throughput of 100 single requests vs one batch of 100 through the same model
The GIL blocks concurrency	CPU utilization plateauing well below 100% while latency climbs
Eager PyTorch/sklearn runtime	Compare against your ONNX Runtime timing from Module 1
No model versioning in the artifact	Explain what happens on a bad deploy with a single <code>model.pkl</code> baked into the image

4. Record the baseline numbers. **Every later category is measured against this row.**

04 CAT 2 — BentoML (the primary web-service deliverable) 2 h

1. Save the model into the Bento store: `bentoml.sklearn.save_model("ride_duration", model, signatures={...})`.
2. Write `serving/bentofile.yaml` and `serving/service.py` using a **Runner** so inference runs in a separate worker process, out from under the API's GIL.
3. **Enable adaptive micro-batching** and configure `max_batch_size` / `max_latency_ms`. Explain the trade-off between these two numbers — it is the clearest example in the course of a latency-versus-throughput dial.
4. Add health/liveness endpoints and expose the built-in metrics endpoint (Module 5 will scrape it).
5. Build and containerize:

```
bentoml build && bentoml containerize ride_duration_service:latest
docker push <user>/ride-duration-bento:0.3.0
```

6. Re-run the same 50-user ramp. **Put the BentoML row next to the FastAPI row** and explain each delta by pointing at the specific mechanism — Runner, micro-batching, or runtime.

ACCEPTANCE CHECK

Measurable throughput improvement over CAT 1 at equal or lower p95, with the mechanism explained.

05 Batch scoring 60 min

1. Write `src/prodml/batch.py`: read a Parquet partition, score in chunks with the ONNX or Bento model, write predictions to Parquet partitioned by date.
2. Score **at least 1 million rows**. Record wall-clock time, peak memory, and **cost per million predictions** (rows/sec → estimated instance-hours).
3. Add it to Airflow as a separate DAG on a nightly schedule.
4. Compare cost per million predictions between the batch job and the same volume through the web service. **The number is usually 50–100× apart — that gap is the entire argument for batch inference.**

06 Streaming inference 75 min

1. Add Redis to Compose. Write a producer pushing trip events onto a **Redis Stream**, and `src/prodml/stream_consumer.py` reading via a consumer group, scoring, and publishing to an output stream.
2. Implement the properties that make streaming hard: consumer groups for parallelism, `XACK` acknowledgement, at-least-once delivery, and a dead-letter stream for poison messages.
3. Measure **end-to-end event latency** (event produced → prediction published), not just model latency.
4. State which of Kafka vs Redis Streams you would pick for this workload, and why.

07 Accelerated runtimes 2 h**7b — CAT 4: ONNX Runtime + OpenVINO (required for everyone)**

1. Serve the ONNX model from Module 1 through `onnxruntime` with graph optimizations enabled; measure.
2. Convert ONNX → OpenVINO IR (`ovc` / Model Optimizer), run through the OpenVINO runtime; measure.
3. Report the speedup over eager and the accuracy delta. Name the **execution provider** you used and what alternatives exist.

7a — CAT 3: TensorRT + Triton (GPU track — Colab acceptable)

1. Build a TensorRT engine from the ONNX model in FP16.

2. Write a Triton `config.pbtxt` with a model repository layout, **dynamic batching** (`preferred_batch_size` , `max_queue_delay_microseconds`), and `instance_group` for device concurrency.
3. Serve, query with `tritonclient` , measure. Map each result to one of the three GPU levers: graph optimization, reduced precision, device concurrency.

7c — CAT 5: vLLM (LLM track — required for final project Track B)

1. Serve a small instruct model (Qwen2.5-0.5B / 1.5B, or TinyLlama on constrained hardware) via `vllm serve` .
2. Query it through the **OpenAI-compatible** endpoint — with the same client code you would use against a hosted API.
3. Measure **TTFT** (time to first token) and **inter-token latency** at 1, 5 and 20 concurrent requests.
4. Explain **PagedAttention**, **continuous batching** and the **KV cache** in your own words, and state why a FastAPI-plus-`transformers` loop cannot compete.

08 Load testing 75 min

1. Write `loadtest/locustfile.py` with realistic tasks: 80% single predictions, 15% batch, 5% `/metadata` , `between(1, 3)` wait times, and a **realistic payload distribution** — not the same row 10,000 times, because caching will lie to you.
2. Ramp to **100 concurrent users** against your BentoML service. Record p50 / p95 / p99, RPS, failure rate, and the saturation point where p95 climbs non-linearly.
3. **Identify the bottleneck with evidence.** Watch CPU, memory and GPU utilization during the run, then state which it is: CPU-bound, memory-bound, GIL-bound, I/O-bound, or batching-bound.
4. Change **one** parameter based on your diagnosis (worker count, `max_batch_size` , or `max_latency_ms`), re-run, and report before/after.
5. Produce the full comparison table — the centrepiece of your Module 3 report:

CATEGORY	RUNTIME	P50	P95	P99	RPS @100	FAILURE %
CAT 1	FastAPI eager					
CAT 2	BentoML + micro-batching					
CAT 4	ONNX Runtime					
CAT 4	OpenVINO INT8					
CAT 3	TensorRT + Triton (<i>if GPU</i>)					

09 Release strategies: canary with automatic rollback 90 min

1. Add **nginx** to Compose as a reverse proxy in front of two serving containers: `model-v1` (stable) and `model-v2` (candidate).
2. Configure weighted upstreams for a **90/10 canary split**:

```
upstream model_backend {
    server model-v1:3000 weight=9;
    server model-v2:3000 weight=1;
}
```

3. Tag responses with the serving version (header or response field) so traffic is attributable.
4. Write `serving/canary_promote.sh` ramping 10% → 25% → 50% → 100%, and `serving/rollback.sh` returning to 100% stable in one command. **Time your rollback and report the number in seconds.**
5. Implement the **automatic guard**: a watcher that polls the canary's p95 latency and error rate and calls the rollback script if p95 exceeds your SLA for two consecutive intervals. Trigger it deliberately by deploying a slow model, and screenshot the automatic rollback.
6. Implement **shadow mode** as well: mirror a copy of live traffic to the candidate with `mirror`, compare predictions offline, and return **only** the stable response to the user. Explain why shadow mode is the safest way to validate a *model* change specifically.
7. Complete the strategy table — Blue/Green, Canary, A/B Test, Shadow: traffic mechanics, risk reduced, cost, and one named company example each.

PULL REQUEST – MODULE 3

Branch: `module-3-serving-release` → **Tag after merge:** `v0.3.0`

Paste into the PR: the Airflow Graph view, the full serving comparison table, the Locust charts, your bottleneck diagnosis, and the rollback timing. **Ask your reviewers to check:** is the load test payload realistic or is it hitting a cache? Is the bottleneck claim actually supported by the utilization evidence? Can they run `rollback.sh` and see traffic return to stable?

OUTPUT – WHAT YOU SHIP

1. An **Airflow DAG** with branching, sensors, XCom, retries and backfill, feeding your MLflow registry
2. A written **inference-pattern decision** grounded in SLAs rather than tool preference
3. The **same model served three ways**: web service (BentoML), batch (1 M+ rows), streaming (Redis consumer group)
4. **Multiple runtime categories benchmarked** — eager, ONNX Runtime, OpenVINO, plus TensorRT/ Triton or vLLM
5. **Load test results to 100 concurrent users**, with a bottleneck diagnosed from evidence and one tuning change measured
6. A **canary release with automatic rollback**, plus shadow mode, running behind nginx
7. A benchmark table you can walk an interviewer through line by line

DEFINITION OF DONE

- ☐ Airflow DAG green end to end with branch, sensor, XCom, retries; backfill executed and explained
- ☐ Pattern decision table written **before** serving code
- ☐ CAT 1 baseline measured; all four problems demonstrated with evidence
- ☐ BentoML service built, containerized, pushed; micro-batching configured
- ☐ Batch job scores ≥ 1 M rows; cost-per-million compared against the web service
- ☐ Streaming consumer with consumer groups, `XACK`, dead-letter path; end-to-end latency measured
- ☐ ONNX Runtime and OpenVINO benchmarked (mandatory); TensorRT/Triton **or** vLLM completed
- ☐ Locust run to 100 users; p50/p95/p99 recorded; saturation point identified
- ☐ Bottleneck named with supporting evidence; one tuning change measured before/after
- ☐ nginx canary at 90/10 with version-attributable responses; automatic rollback triggered and screenshotted; rollback time recorded
- ☐ Shadow mode implemented; release strategy comparison table complete
- ☐ `reports/module-3.md` complete; PR reviewed and merged; `v0.3.0` tagged

FAILURE MODES

MISTAKE	COST
Passing DataFrames through XCom	Airflow metadata DB bloat and mysterious task failures
Load testing with one identical payload	Every cache in the stack lies to you; results are meaningless
Reporting mean latency instead of p95/p99	You optimize for the users who were already fine
Running the load test on the same laptop as the service	You measured your laptop, not your service
Setting <code>max_batch_size</code> very high to "go faster"	p99 latency explodes; throughput and latency are a dial, not a switch
A canary with no rollback trigger	A slow-motion outage you watch happen
Skipping Step 2 and opening the BentoML docs first	The most common architectural failure in the industry, reproduced at small scale

RESOURCES – MODULE 3

PROVIDED BY THE INSTRUCTOR

- Airflow: youtu.be/vEApEfa8HXk

ORCHESTRATION

- Apache Airflow docs: airflow.apache.org/docs · Best practices: [airflow best practices](https://airflow.apache.org/airflow-best-practices)
- Astronomer guides — the best practical writing on DAG design: astronomer.io/docs/learn
- Prefect — the common alternative you will be asked about: docs.prefect.io

SERVING

- BentoML: docs.bentoml.com
- NVIDIA Triton: docs.nvidia.com — Triton · model configuration & dynamic batching: [config.pbtxt reference](https://docs.nvidia.com/triton-inference-server/config-pbtxt/reference)
- ONNX Runtime performance tuning: onnxruntime.ai/docs/performance · OpenVINO: docs.openvino.ai
- vLLM: docs.vllm.ai · Ray Serve (worth knowing exists): docs.ray.io/serve

STREAMING, LOAD TESTING, RELEASE

- Redis Streams: redis.io — streams · Kafka quickstart: kafka.apache.org/quickstart
- Locust: docs.locust.io · k6, good for CI-integrated load tests: grafana.com/docs/k6
- nginx load balancing: nginx.org — load balancing
- Google SRE Workbook — *Canarying Releases*: sre.google/workbook/canarying-releases

DEEPER READING

- Chip Huyen, *Designing Machine Learning Systems* — Chapter 7 covers deployment patterns better than any blog post: huyenchip.com/books
- Chip Huyen, *Batch vs online prediction*: huyenchip.com — real-time ML

04

MODULE 4 · MODEL OPTIMIZATION – FASTER, SMALLER, CHEAPER · 11 LESSONS

Mini Project 4 — The optimization journey

Goal: take one trained model through every optimization technique in the module, measuring accuracy, latency, throughput and size at each stage — producing one journey table that quantifies exactly what each technique costs and buys. Then deploy the winner back into the Module 3 serving stack and prove the improvement.

Session coverage map

LESSON	WHERE IT APPEARS
1 — Why optimization matters	Step 1 — baseline plus a cost model in dollars
11 — Benchmarking and the decision framework	Step 1 — build the harness <i>first</i> ; Step 10 — the decision writeup
2 — Pruning	Step 3 — 30/50/70% sparsity sweep with fine-tuning
3 — Post-training quantization	Step 4 — dynamic vs static, calibration set size
4 — Quantization-aware training	Step 5 — QAT, and the explicit verdict vs PTQ
5 — Knowledge distillation	Step 6 — teacher → student with temperature scaling
6 — TensorRT	Step 7 — FP16 and INT8 engine builds
7 — ONNX Runtime	Step 8 — execution providers, dynamic axes
8 — OpenVINO	Step 8 — IR conversion + INT8 POT
9 — Edge deployment	Step 9a — TFLite / GGUF / Core ML with on-device validation
10 — LLM-specific quantization	Step 9b — AWQ / GPTQ and the VRAM table

PREREQUISITES

Completed: Mini Projects 1–3. You need a trained model, a working ONNX export, and the Module 3 benchmark table — its numbers are Row 0 of this project.

Knowledge: what FP32/FP16/INT8 mean at the bit level; forward vs backward pass; that "accuracy" means MAE/RMSE for regression and top-1/F1 for classification.

Machine: CPU-only is fine for most of it — pruning, PTQ, QAT, distillation, ONNX Runtime, OpenVINO and TFLite all run on a laptop. **TensorRT and AWQ/GPTQ need an NVIDIA GPU** — use Colab (free T4) or Kaggle (free P100/T4, 30 h/week), save the notebook into `optimization/`, and record which hardware produced each number.

Never compare a Colab T4 number against a laptop CPU number in the same column — add a `hardware` column to the table. **Time:** 8–10 hours.

A NOTE ON WHICH MODEL TO USE

A small tabular regressor will not show interesting quantization behaviour — INT8 on a linear model is a rounding error. Use a model with real depth: a small CNN (ResNet-18 on CIFAR-10) or a small transformer. **This is the natural place to bring in your final project model** — AraBERT for Track A, or your generation model for Track B. Everything you measure here goes straight into your final project report.

Steps

01 Build the benchmark harness before optimizing anything 90 min

This is not preparation for the project. It is the most valuable part of it. An unmeasured optimization is a rumour.

1. Write `optimization/benchmark.py` exposing one function:

```
def benchmark(model_fn, dataset, name: str, hardware: str) -> BenchmarkResult
```

2. It must produce, every time, for every variant: **accuracy** (task metric on the *same* held-out set, plus delta vs baseline) · **latency** (p50, p95, p99 at batch 1 and batch 32) · **throughput** (samples/sec at your service's real batch size) · **size** (on-disk MB) · **memory** (peak RSS, peak VRAM) · **utilization** (GPU% via `nvidia-smi` / `pynvml`, else CPU%).
3. Correctness requirements — get these wrong and every number in the module is wrong:
 - **Warmup** — discard the first 50 iterations (lazy CUDA init, JIT, cache warming)
 - **Repetition** — ≥ 500 timed iterations
 - **Synchronization** — call `torch.cuda.synchronize()` before stopping the timer on GPU. Without it you are timing kernel *launch*, not execution
 - **Fixed seeds** and a fixed evaluation set across every variant
 - **One variable at a time** — never change batch size and precision in the same measurement
4. Append every result to `optimization/results.csv` and render it to Markdown for the report.
5. Run it on your unoptimized model. That is Row 0.

6. Write the **cost model**: given your throughput, how many instances do you need for 1,000 req/s, and what does that cost per month at a real cloud price? Every later row converts into a dollar figure.

ACCEPTANCE CHECK

Running the harness twice on the same model gives numbers within 5% of each other. If not, your measurement is noise and nothing downstream is meaningful.

02 Establish the baseline row 30 min

Run the harness on eager PyTorch/sklearn at batch 1 and batch 32, on every hardware target you have. Then write down, **before** optimizing, which single metric you are optimizing for and what your constraint is — e.g. *"minimize p95 latency subject to accuracy loss < 1%."* Optimization without a stated objective is drifting.

03 Pruning 75 min

1. Apply **magnitude-based unstructured pruning** with `torch.nn.utils.prune` at **30%, 50% and 70%** sparsity. Benchmark each.
2. Add a **fine-tuning pass** after each round (iterative pruning) and re-benchmark. Report the accuracy recovered by fine-tuning as a separate number — usually the difference between "unusable" and "shipped".
3. Call `prune.remove()` to make pruning permanent, then check file size. **Explain why unstructured pruning often does not shrink the artifact or speed up inference** — sparse weights are still stored densely and generic kernels do not skip zeros.
4. Compare against **structured pruning** (removing whole channels/heads) on at least one layer, and explain which actually yields a wall-clock speedup.
5. State the **practical sparsity ceiling** you found, with the accuracy curve to support it.

04 Post-training quantization 75 min

1. Apply **dynamic quantization** (weights INT8, activations quantized at runtime). Benchmark.
2. Apply **static quantization** with a calibration set. Use ~300 representative samples — then deliberately test with 30 and with 3,000, and report how accuracy differs. **Discovering empirically that 300 is enough is the lesson.**
3. Do the same via **ONNX Runtime's quantizer** (`quantize_dynamic`, `quantize_static`) and compare with the PyTorch path.
4. Record the accuracy delta. For most classification models this lands **under 0.5% top-1** — if yours is much worse, find the responsible layer with per-layer sensitivity analysis and say so.
5. Explain where calibration data must come from — a production-like distribution, not the training set's easiest rows — and what happens if it is unrepresentative.

05 Quantization-aware training 75 min

1. Implement the PyTorch QAT workflow: `prepare_qat` with fake-quant observers → fine-tune a few epochs → `convert`.
2. Benchmark and compare directly against your PTQ row.
3. Record the **training cost**: GPU-hours and wall-clock for QAT versus the minutes PTQ took.
4. Write the verdict explicitly: *given my accuracy gap of X% and my training cost of Y hours, QAT is / is not worth it for this model*. Then state the general rule you would apply at work — and the case where the answer flips.

06 Knowledge distillation 90 min

1. Pick a **student** roughly 3–5× smaller than your teacher and justify the size choice in a paragraph.
2. Implement the distillation loss:

$$L = \alpha \cdot \text{KL}(\text{student_soft} \parallel \text{teacher_soft}) \cdot T^2 + (1-\alpha) \cdot \text{CE}(\text{student_hard}, y_{\text{true}})$$

with **temperature T** applied to both softmaxes.

3. Sweep $T \in \{1, 3, 5, 10\}$ and $\alpha \in \{0.3, 0.7, 0.9\}$, at minimum a small grid. Report which combination transferred best.
4. Benchmark the student. Also train **the same student architecture from scratch on hard labels only** — the gap between "student with distillation" and "student alone" is the actual measurement of knowledge transfer, and most write-ups omit it. Yours will not.
5. Connect to the session's real examples (Google Translate 600M→77M; MobileBERT) and state your compression ratio at what accuracy cost.

07 TensorRT compilation 60 min · GPU

1. Convert ONNX → TensorRT engine in **FP32**, **FP16** and **INT8** (INT8 uses the calibrator from Step 4).
2. Benchmark all three. Explain each speedup by naming the mechanism: **layer fusion**, **precision reduction**, **kernel auto-tuning**, **memory planning**.
3. Record engine build time separately — it is minutes, and it matters for CI.
4. Document the **portability limitation**: an engine is tied to a GPU architecture and TensorRT version. State what that means for a CI pipeline that builds on one machine and deploys on another. This is the constraint that surprises teams in production.

08 Cross-platform and CPU: ONNX Runtime + OpenVINO 60 min

1. Re-export to ONNX with **correct dynamic axes** on the batch dimension and verify with `onnx.checker`. Show what breaks when they are wrong — a model that only accepts batch size 1.
2. Benchmark ONNX Runtime with graph optimizations at `basic`, `extended` and `all`.
3. Benchmark across **execution providers** available to you: `CPUExecutionProvider`, `CUDAExecutionProvider`, `OpenVINOExecutionProvider`, `TensorrtExecutionProvider`.

4. Convert ONNX → **OpenVINO IR**, apply INT8 post-training optimization, benchmark. Note whether your CPU supports AVX-512 (`lscpu | grep avx512`) and how that shows up.
5. State the rule you would give a teammate: **which runtime is the right default for a CPU-only cloud fleet, and why.**

09 Edge and LLM-specific compression 90 min

9a — Edge deployment (pick at least one, validate on-device or in an emulator)

- **TFLite / LiteRT** — convert, apply INT8 with a representative dataset, report size and latency on Android or an ARM emulator
- **GGUF + llama.cpp** — quantize to Q8_0, Q5_K_M and Q4_K_M; report size and tokens/sec per level
- **Core ML** for Apple silicon • **NVIDIA Jetson** if you have the hardware

For each level report **size**, **latency**, and a **quality check on real inputs**. A model that fits on the device but returns nonsense is not a deployment.

9b — LLM-specific quantization

1. Explain in your own words why classical symmetric INT8 breaks on LLMs: **activation outliers**, attention-pattern sensitivity, the weight distribution problem.
2. Apply **AWQ** or **GPTQ** 4-bit to a small model (Qwen2.5-1.5B or similar) and measure perplexity or task accuracy before and after.
3. Produce the **VRAM table**: FP16 / INT8 / 4-bit → VRAM required, tokens/sec, quality delta. Then compute what it takes to run a 70B model on a single 80 GB GPU, showing the arithmetic.

10 The journey table, the decision framework, and redeployment 75 min

1. Assemble the **complete journey table** in `reports/module-4.md` :

#	TECHNIQUE	ACCURACY	Δ ACC	P95 (BS=1)	THROUGHPUT	SIZE MB	HARDWARE
0	Baseline (eager)		—				
1	TorchScript						
2	ONNX export						
3	Pruning 50% + fine-tune						
4	PTQ INT8						
5	QAT INT8						
6	Distilled student						

#	TECHNIQUE	ACCURACY	Δ ACC	P95 (BS=1)	THROUGHPUT	SIZE MB	HARDWARE
7	TensorRT FP16						
8	TensorRT INT8						
9	OpenVINO INT8						
10	TFLite / GGUF (edge)						

1. Write the **optimization decision framework**: a decision tree keyed on your bottleneck. *If GPU-bound → try X first. If size-constrained for edge → Y. If accuracy is non-negotiable → Z. If cost per million predictions is the problem → W.*
2. Answer "**when do I stop?**" in one paragraph, with a concrete stopping rule tied to your SLA and cost model.
3. **Redeploy**. Put the winning artifact behind the Module 3 serving stack, re-run the same Locust test, and place the new p95/RPS beside the Module 3 numbers. **The optimization is only real when it appears in the serving benchmark.**
4. Convert the improvement into money: *"this saves \$N/month at 1,000 req/s."* That sentence is what gets an optimization approved at work.

PULL REQUEST – MODULE 4

Branch: `module-4-optimization` → **Tag after merge:** `v0.4.0`

Paste into the PR: the full journey table with its `hardware` column, the decision framework, and the before/after serving benchmark. **Ask your reviewers to check:** does the harness include warmup and CUDA synchronization? Is every row measured on the same evaluation set? Is any row comparing across different hardware without saying so?

OUTPUT – WHAT YOU SHIP

1. A **reusable benchmark harness** with correct warmup, synchronization and fixed seeds — the most portable artifact in this entire course
2. A **model taken through 10+ optimization stages**, each measured on accuracy, latency, throughput, size and memory
3. **The journey table** — one artifact quantifying the whole optimization space for your model
4. Empirical answers to what everyone else guesses at: the sparsity ceiling, the calibration-set size, whether QAT is worth it, how much distillation actually transfers
5. **Edge and LLM compression** results with on-device or quality validation
6. A written **decision framework** and a stated stopping rule

7. The optimized model redeployed and re-benchmarked in the serving stack, with the saving in dollars per month

DEFINITION OF DONE

- ☐ Harness reproducible within 5% across runs; warmup and CUDA sync implemented
- ☐ Baseline row recorded before any optimization; objective and constraint stated
- ☐ Pruning at 30/50/70% with and without fine-tuning; ceiling reported; structured vs unstructured explained
- ☐ PTQ dynamic and static; calibration-size experiment (30 / 300 / 3000) run and reported
- ☐ QAT implemented; training cost recorded; explicit verdict vs PTQ
- ☐ Distillation with T and α sweep; student-from-scratch control included
- ☐ TensorRT FP16 and INT8 (or documented Colab equivalent); portability limitation explained
- ☐ ONNX Runtime across ≥ 2 execution providers; OpenVINO IR + INT8
- ☐ At least one edge format with quality validation; AWQ or GPTQ 4-bit with a VRAM/quality table
- ☐ Complete journey table with a `hardware` column; decision framework and stopping rule written
- ☐ Optimized model redeployed and re-load-tested against Module 3 numbers; monthly cost saving computed
- ☐ `reports/module-4.md` complete; PR reviewed and merged; `v0.4.0` tagged

FAILURE MODES

MISTAKE	COST
Benchmarking without warmup	The first CUDA call dominates; your "speedup" is initialization noise
Forgetting <code>torch.cuda.synchronize()</code>	You timed kernel launches. Numbers can be 10× too optimistic
Comparing across different hardware in one table	A conclusion that is simply false
Measuring latency but never accuracy	You shipped a fast wrong model
Calibrating on training data	Great offline numbers, quantization collapse in production
Expecting unstructured pruning to speed things up	Confusion; the explanation is the deliverable, not the speedup

MISTAKE**COST**

Optimizing without a stated objective

You will not know when to stop

RESOURCES – MODULE 4**PRUNING AND QUANTIZATION**

- PyTorch pruning tutorial: [pytorch.org — pruning](https://pytorch.org/pruning)
- PyTorch quantization: pytorch.org/docs/quantization · static & QAT tutorial: [pytorch.org — static quantization](https://pytorch.org/static-quantization)
- ONNX Runtime quantization: [onnxruntime.ai — quantization](https://onnxruntime.ai/quantization) · Hugging Face Optimum: huggingface.co/docs/optimum

DISTILLATION

- Hinton, Vinyals & Dean, *Distilling the Knowledge in a Neural Network* (2015): arxiv.org/abs/1503.02531
- Sanh et al., *DistilBERT* (2019): arxiv.org/abs/1910.01108
- Hugging Face distillation guide: [huggingface.co — knowledge distillation](https://huggingface.co/knowledge-distillation)

COMPILERS AND RUNTIMES

- NVIDIA TensorRT: docs.nvidia.com/tensorrt · Torch-TensorRT: pytorch.org/TensorRT
- OpenVINO optimization guide: [docs.openvino.ai — optimize inference](https://docs.openvino.ai/optimize-inference)

EDGE

- LiteRT (formerly TensorFlow Lite): ai.google.dev/edge/litert
- llama.cpp and GGUF: github.com/ggml-org/llama.cpp · Core ML Tools: apple.github.io/coremltools
- NVIDIA Jetson: developer.nvidia.com/embedded

LLM QUANTIZATION

- Lin et al., AWQ (2023): arxiv.org/abs/2306.00978 · Frantar et al., GPTQ (2022): arxiv.org/abs/2210.17323
- Dettmers et al., *LLM.int8()* — the outlier problem explained (2022): arxiv.org/abs/2208.07339
- Hugging Face quantization overview: [huggingface.co — quantization](https://huggingface.co/quantization)

05

MODULE 5 · OBSERVABILITY AND DRIFT DETECTION · 16 LESSONS

Mini Project 5 — The observable ML system

Goal: add a full monitoring layer to your deployed service — a real Prometheus endpoint, resource monitoring across CPU/RAM/disk/GPU, a four-row Grafana dashboard provisioned from your repo, at least six alert rules with Alertmanager routing, a scheduled Evidently suite writing to PostgreSQL, LLM observability if your project has an LLM component, and a runbook an on-call engineer who has never seen your code can follow at 3am.

Session coverage map

LESSON	WHERE IT APPEARS
1 — Why production models degrade	Step 1 — the drift simulator and the four failure modes
2 — Drift taxonomy	Step 1 — taxonomy table, drift dynamics, three-tier monitoring strategy
3 — Statistical detection methods	Step 2 — implement and unit-test all six, with a detection matrix
4 — Prometheus fundamentals	Step 3 — the metric contract, written before any code
5 — Instrumenting a model service	Step 4 — service metrics, the multiprocess trap, the exporter zoo, scrape config
6 — PromQL for ML	Step 5 — the queries that answer real questions, plus recording rules
7 — Grafana dashboards as code	Step 6 — four-row layout, provisioned from the repo, deploy annotations
8 — Alerting people trust	Step 7 — six rules, routing, symptom vs cause, SLO burn-rate
9 — Case study 1: monitoring a live vision model	Step 8 — the GPU and host layer, DCGM, the throttling diagnosis
10 — Case study 2: inside the service	Step 9 — stage timing, pynvml, server internals, label-free quality signals
11 — Drift reports and the metrics store	Step 10 — Evidently → PostgreSQL → Grafana → CI gate → Airflow retrain branch
12 — Logs, metrics and traces: the ELK heads-up	Step 11 — the prediction-event schema that joins a log line to a metric
13 — LLM observability with Langfuse	Step 12 — traces, spans, generations, scores, self-hosting, @observe

LESSON	WHERE IT APPEARS
14 — Prompt management, cost and latency	Step 13 — prompts as versioned config, cost per trace, TTFT and tokens/sec
15 — Evaluation without ground truth	Step 14 — your application's scorecard, LLM-as-a-judge, RAGAS, the CI gate
16 — Self-hosted LLMs, guardrails and rate limits	Step 15 — vLLM internals, input/output guardrails as metrics, token bucket

HOW TO SCOPE THIS MODULE

This is the widest module in the course — 16 lessons across two halves. **Steps 1–11 and Step 16 are mandatory for everyone.**

Steps 12–15 (the LLM half) are required if your final project has an LLM component — Track B, or Track A if you added any generative feature. If your project is pure classical ML, do Step 12 at minimum on a small toy RAG pipeline so you have seen a trace; the rest of the LLM half is then optional but strongly recommended, because it is the half of the job market that is growing.

PREREQUISITES

Completed: Mini Projects 1–4. Specifically you need: a serving layer to instrument (M3), an Airflow instance to schedule detection in (M3), a PostgreSQL instance already running in your Compose stack (M2, the MLflow backend — you will reuse it), and the `repository_dispatch` CT trigger you left wired in Module 2. **Step 10 fires into it.**

Knowledge: what a distribution is; what a p-value means and its limitation at large sample sizes; HTTP endpoints and scraping; basic SQL (`CREATE TABLE` , `INSERT` , `SELECT ... GROUP BY`); basic RAG architecture (retriever → context → generator) for the LLM half.

Access: a free Slack workspace with an incoming webhook (or Discord) for Alertmanager. For the LLM half: **self-hosted Langfuse** via Docker Compose (free, no account), plus either your Module 3 vLLM endpoint or a small amount of API credit. **Prefer the self-hosted, local-model path** — it costs nothing and is closer to what regulated employers in the region actually run.

Machine: 8+ GB RAM. **GPU optional** — Steps 8 and 9 are written for GPU serving, and every one has a documented CPU-only substitute so nobody is blocked. **Time:** 12–16 hours. Start the day after the session.

Part A — Drift foundations

01 Build a drift simulator, write the taxonomy, choose your tier 75 min

You cannot monitor drift you cannot cause. Build it deliberately.

1. Write `monitoring/simulate_drift.py` producing corrupted versions of your reference data covering the four **types**: **data drift** (shift a numeric feature 30%, change the categorical mix) · **concept drift** (same features, different relationship) · **label drift** (shift the target distribution) · **embedding drift** (shift the semantic content of a text field).
2. Then produce the four **dynamics** for at least one of those types: **sudden** (step change at day 30), **gradual** (linear over 60 days), **incremental** (small repeated shifts), and **recurring** (a seasonal pattern that returns). The dynamic decides your detector's window and threshold as much as the type does.
3. Write the **taxonomy table** for your own system:

DRIFT TYPE	WHAT CHANGES	FIRST SIGNAL TO APPEAR	GROUND TRUTH NEEDED?	TOOL	REAL EXAMPLE
Data drift	$P(X)$				Uber during COVID
Concept drift	$P(Y X)$				Twitter sentiment post-2020
Label drift	$P(Y)$				PayPal fraud rate during promotions
Embedding drift	semantic space				Notion AI after launch

1. Add one column of your own: "**how long until a user notices**". That column is the entire justification for this module — none of these throw an error, and models degrade for weeks before anyone looks.
2. Write your **three-tier monitoring strategy** in `reports/module-5.md`: which signals you watch *continuously* (cheap, every request), which you compute *on a schedule* (expensive, daily or weekly), and which you only look at *during an investigation*. You cannot afford to watch everything. Deciding what you will not watch is the strategy.

02 Implement the detection methods yourself 100 min

Before using any library, implement each in `monitoring/drift_stats.py` with a unit test:

METHOD	USE FOR	TEST IT MUST PASS
KS test	continuous features	fires on shifted, silent on identical

METHOD	USE FOR	TEST IT MUST PASS
Chi-square	categorical features	fires on a changed category mix
Wasserstein distance	magnitude of a shift, not just its presence	the value scales with how far the distribution moved
JS / KL divergence	probability and prediction distributions	KL asymmetry shown: $KL(P Q) \neq KL(Q P)$; JS is symmetric and bounded
MMD + domain classifier	high-dimensional embeddings	MMD fires on shifted embeddings; the classifier's AUC $\approx$ 0.5 on the control and rises with drift
ADWIN / DDM	streaming error, concept drift	detects a step change; detection delay reported in samples

1. **The domain classifier is the one to understand deeply.** Train a binary classifier to tell reference embeddings from current embeddings. If it cannot (AUC $\approx$ 0.5), there is no drift. If it can (AUC $\rightarrow$ 1.0), there is — and its feature importances tell you *what* drifted. Report the AUC on your control and on each drifted set.
2. For **each** method write one sentence on **what it costs and what it misses**. In particular: **p-value tests fire on statistically significant but practically meaningless differences once n is large**, which is why high-traffic teams move to magnitude measures like Wasserstein. And per-dimension tests on a 384-dimensional embedding collapse under multiple comparisons — that is why MMD and domain classifiers exist.
3. Run every method against every simulated drift type and dynamic, and produce a **detection matrix**: which method caught which drift, and after how many samples.
4. Run **ADWIN and DDM** over a stream of prediction errors across at least three parameter settings each, and report the sensitivity versus false-alarm trade-off.
5. Address the **ground truth lag problem** explicitly: your true labels arrive hours or days late. Write what your monitoring watches *during the gap* — and connect it to the label-free quality signals in Step 9.

ACCEPTANCE CHECK

The detection matrix is filled in, and

NO METHOD FIRES ON THE UNDRIFTED CONTROL.

A detector with no measured false-positive rate is not a detector.

Part B — The metrics stack

03 Design the metric contract before writing any code 45 min

Prometheus **pulls**. Your service does not push anywhere — it exposes `/metrics` and Prometheus scrapes it on an interval. That one design fact changes everything downstream: your metrics must be cheap to read, valid at any instant, and survive being scraped by more than one thing.

1. Write the **metric contract** in `reports/module-5.md` before touching code. For every metric: name, type, labels, and one sentence on why that type:

METRIC	TYPE	WHY THIS TYPE
<code>predictions_total{model_version,status}</code>	Counter	monotonically increasing event count — you query its <code>rate()</code> , never its value
<code>prediction_latency_seconds{stage}</code>	Histogram	you need quantiles across instances, which an average cannot give you
<code>prediction_score</code>	Histogram	the output distribution over time — your cheapest continuous drift signal
<code>inference_batch_size</code>	Histogram	effective batching, the number that explains your throughput
<code>inflight_requests</code>	Gauge	a value that goes up and down; saturation
<code>model_info{version,framework}</code>	Gauge (info)	which version is serving right now, joinable onto every other query

2. Follow the **naming conventions**: `snake_case`, a namespace prefix, base units (**seconds and bytes, never milliseconds or megabytes**), and `_total` on counters. Grafana formats units; your metric should not.
3. Write your **cardinality budget**. Series = product of all label values. Compute it for your design and write the number down. Then state which labels you deliberately did *not* add — `user_id`, `request_id`, `correlation_id`, raw input text. **Unbounded label values are the single most common way engineers take down a Prometheus cluster.** Those belong in logs and traces, which is exactly what Steps 11 and 12 are for.
4. Choose your **histogram buckets deliberately**, around your actual SLA — e.g. `[.005, .01, .025, .05, .1, .25, .5, 1, 2.5]`. Default buckets will hide your tail. Justify your choice in one sentence.
5. `curl` the raw `/metrics` output of any exporter and **read the exposition format by eye** — `HELP`, `TYPE`, the `_bucket` / `_sum` / `_count` triplet a histogram actually expands into. Paste ten lines into your report and annotate them. Everything else in this stack is built on this text format.

04 Instrument the service and deploy the exporters 2 h

1. Instrument your serving layer with `prometheus-client`, implementing every metric in your contract. Time **each pipeline stage separately**, not just the total — a single end-to-end latency number tells you that you are slow, never where.
2. **Handle the multiprocess trap.** Under gunicorn or multiple uvicorn workers, each process keeps its own registry, so a scrape hits one worker at random and your counters appear to jump backwards. Set `PROMETHEUS_MULTIPROC_DIR` and use `MultiProcessCollector`. **Prove it:** run with 4 workers, scrape repeatedly, and show the before/after in your report. This bug is silent, it is common, and every number above it is wrong until you fix it.
3. Deploy the **exporter zoo** — at minimum:

EXPORTER	GIVES YOU
<code>node_exporter</code>	host CPU, RAM, disk, network — mandatory
<code>cAdvisor</code>	per-container CPU, memory, throttling — mandatory
DCGM exporter	GPU metrics — if you serve on GPU
Triton / vLLM built-in <code>/metrics</code>	inference server internals — if you use them
blackbox exporter	probes your endpoint from outside — the only one that tells you the service is down when the service is down
kube-state-metrics	cluster object state — document it even if you are not on Kubernetes

4. Write `monitoring/prometheus.yml` yourself: global scrape interval, one job per target, and **at least one `relabel_config`** — rewrite a label, drop a noisy metric with `metric_relabel_configs`, or attach a static `env` label. Explain in one sentence what relabeling is for and when service discovery replaces a static target list.
5. Confirm every target is **UP** in the Prometheus targets page and screenshot it.

ACCEPTANCE CHECK

Your `/metrics` endpoint exposes real values that change under load, counters do not go backwards with 4 workers, and every scrape target is UP.

05 PromQL that answers real questions 90 min

Write these yourself, run them, screenshot the graph, and state in one line what each answers. **Do not paste them from the slides — change them to match your own labels.**

```
# p95 latency by model version
histogram_quantile(0.95,
  sum(rate(prediction_latency_seconds_bucket[5m])) by (le, model_version))

# error rate during a canary, split by version
sum(rate(predictions_total{status="error"}[5m])) by (model_version)
  / sum(rate(predictions_total[5m])) by (model_version)

# has the prediction mix shifted over 24h?
sum(increase(prediction_score_bucket[24h])) by (le)

# queue time as a share of total latency – is the model slow, or is it waiting?
sum(rate(stage_latency_seconds_sum{stage="queue"}[5m]))
  / sum(rate(prediction_latency_seconds_sum[5m]))

# throughput per second, and effective batch size
sum(rate(predictions_total[1m]))
histogram_quantile(0.5, sum(rate(inference_batch_size_bucket[5m])) by (le))
```

1. Understand the distinction you will be asked about in interviews: **rate()** is per-second and for graphing; **increase()** is a total over a window and for reporting. Both handle counter resets; the raw counter value is almost never what you want.
2. Add at least **three recording rules** in `monitoring/rules/recording.yml` for the expensive queries your dashboard repeats. Show the panel load time before and after — that difference is why recording rules exist.
3. Write one paragraph on **retention and long-term storage**: what your local retention is set to, why raw high-cardinality data cannot be kept for a year, and what **Thanos** or **Mimir** would add. You do not have to deploy either — you have to be able to explain when you would.

06 Grafana dashboards as code 90 min

1. Build a dashboard with the **four-row layout** for an ML service:

ROW	PANELS
1 • Service health	request rate, error rate, latency p50/p95/p99, in-flight requests, uptime from the blackbox probe
2 • Resources	CPU, RAM, disk, network from node_exporter; per-container from cAdvisor; GPU utilization, VRAM, temperature and power if applicable
3 • Model behaviour	prediction score distribution over time, class or output mix, effective batch size, stage latency breakdown, currently serving version
4 • Drift history	the drift scores from Step 10, read from PostgreSQL — the long-horizon row

2. **Provision it from your repo.** Put the datasource YAML and the dashboard JSON under `monitoring/grafana/provisioning/` and mount them in Compose so a fresh `docker compose up` comes up with the dashboard already there. **A dashboard you clicked together in the UI is a dashboard you will lose** — and your reviewer cannot see it at all.

3. Add **deploy annotations**: a vertical line on every graph whenever you ship a new model version. Half of all "why did latency change at 14:20?" questions answer themselves the moment deploys are on the same axis as metrics.
4. Use template variables (`$model_version` , `$instance`) so one dashboard serves every version rather than one dashboard per model.

ACCEPTANCE CHECK

Delete your Grafana container and volume, run `docker compose up`, and the dashboard is back with no clicking.

07 Alerting people trust 90 min

Write **at least six alert rules** in `monitoring/rules/alerts.yml` with Alertmanager routing to Slack. Suggested set:

ALERT	SEVERITY	SYMPTOM OR CAUSE?
Service down (blackbox probe failing)	critical / page	symptom
p95 latency above SLA for 5 min	critical / page	symptom
Error rate above 1% for 5 min	critical / page	symptom
Drift score above threshold on any monitored feature	warning / ticket	cause
GPU VRAM above 90%, or container memory near its limit	warning / ticket	cause
Prediction volume dropped > 50% versus the same hour yesterday	warning / ticket	symptom
Model artifact age exceeds your retraining SLA	info	cause

1. Use `for:` on every rule so a single scrape blip never pages anyone.
2. Configure Alertmanager **routing, grouping and inhibition**: critical to a paging channel, warnings to a ticket channel; group by service; and inhibit the downstream latency alert when the "service down" alert is already firing. **One incident should produce one notification, not fifteen.**
3. **Symptom versus cause**. Page on symptoms — things users feel. Ticket on causes — things that will hurt later. Label every one of your rules in the table above and defend any you classified differently.
4. Implement **one multi-window SLO burn-rate alert**: define an SLO (e.g. 99% of requests under 200 ms over 30 days), then alert when a fast window *and* a slow window are both burning error budget. Explain in two sentences why two windows beat a single threshold — the short window catches fast burns, the long window suppresses the noise.

5. Every alert's annotation must contain an **obvious first action** and a link to the runbook section from Step 16. An alert that says "drift detected" and nothing else gets muted by week two.
6. Trigger at least three alerts deliberately and screenshot the Slack messages.

Part C — The case study, replicated on your own service

NO GPU? NOT BLOCKED.

Steps 8 and 9 follow the session's real-time vision case study — a defect-detection model on two A10G GPUs with an 80 ms budget and no labels. If you have no GPU, run the same steps against your **CPU** service: replace DCGM metrics with node_exporter and cAdvisor, replace `pynvml` with `psutil`, and replace the throttling diagnosis with a CPU-throttling one (cAdvisor's `container_cpu_cfs_throttled_seconds_total` is the direct analogue). The reasoning you are learning — reading several resource signals *together* to name one cause — is identical. Say in your report which path you took.

08 Case study part 1 — the GPU and host layer 90 min

1. Deploy the **DCGM exporter** (DaemonSet on Kubernetes, or a Compose service locally) and get GPU metrics into Prometheus. CPU path: node_exporter + cAdvisor, already running from Step 4.
2. Build the **resource row** with the metrics that actually matter, and write what each one tells you:

SIGNAL	WHAT IT ACTUALLY MEANS
GPU utilization vs <code>SM_ACTIVE</code>	Utilization only says a kernel was resident. SM_ACTIVE says the chip was doing work. 95% utilization at 20% SM_ACTIVE is a starving GPU, not a busy one
Framebuffer memory used / free	Your OOM predictor. Alert on the trend, not the crash
Temperature + SM clock together	Clock falling while temperature is high = thermal throttling
Power draw + SM clock together	Clock falling while power is pinned at the limit = power capping
PCIe throughput	High transfer with low SM_ACTIVE = you are feeding the GPU too slowly
ECC errors	Rare, but correctable errors climbing means the card is going

3. **Write the three-way diagnosis explicitly.** Given a drop in throughput, how do you tell **thermal throttling** from **power capping** from a **starving GPU**? Each has a different fix — cooling, power budget, and input pipeline respectively — and no single metric distinguishes them. This is the most transferable thing in the module.

4. Add host CPU, RAM, disk and network PromQL for the machine. Include disk — **a full disk takes down more ML services than a bad model does.**
5. Load-test with Locust from Module 3 while watching this row, and screenshot the resource dashboard under load.

09 Case study part 2 — inside the service 2 h

1. **Instrument the pipeline stage by stage.** For a vision pipeline that is decode → preprocess → infer → NMS → post; for your own service it is whatever the equivalent stages are. Use one histogram with a `stage` label. Then answer, from the dashboard alone: **which stage owns your p95?** It is very often not the model.
2. Read **GPU state from inside your own process** with `pynvml` (or `psutil` on CPU) and expose it as a metric, so you have your process's view alongside the exporter's node-level view. Explain when the two disagree and why that matters on a shared machine.
3. If you serve through **Triton or vLLM**, scrape their internal metrics and put them on the dashboard: **queue duration** vs **compute time** (the single most diagnostic pair in model serving — it tells you whether to add replicas or make the model faster), **effective batch size**, and **cache hit rate**.
4. **Model-quality signals without labels.** This is the heart of the case study: the defect model has no ground truth at request time, so quality is inferred from the output and the input. Implement at least four for your own model:
 - **Output signals** — detections per frame (or predictions per request), the confidence distribution, class mix, empty-result rate, and **near-threshold share** (predictions sitting just either side of your decision boundary — the first thing to move when a model starts losing its grip)
 - **Input signals** — for vision: brightness and blur. For tabular: null rate and out-of-range share. For text: length distribution and out-of-vocabulary rate
5. Build the **symptom → cause diagnosis table** for your service. For each symptom, list the candidate causes and the one metric that discriminates between them:

SYMPTOM	CANDIDATE CAUSES	METRIC THAT DISCRIMINATES
p95 latency up, throughput flat	queueing · thermal throttle · bigger inputs	queue duration vs compute time; SM clock; input size histogram
Detections per frame drops	real drift · camera or upstream change · threshold change	input brightness/blur; near-threshold share; deploy annotation
GPU utilization high, throughput low	starving GPU · batch size 1 · preprocessing bound	SM_ACTIVE; effective batch size; stage latency
Error rate up after a deploy	bad model · schema change · dependency	error rate by <code>model_version</code> ; validation rejection counter

6. Do **USE-based capacity planning**: for each resource record Utilization, Saturation and Errors, then answer with numbers — how many requests per second can this instance take before p95 breaks its SLA, and what do you need for 10× that? Cite the Locust run that supports it.

ACCEPTANCE CHECK

You can point at the dashboard and name which stage owns your p95, and your symptom → cause table has a discriminating metric for every row.

Part D — The metrics store and the logs

10 Drift reports, PostgreSQL, and closing the loop 2 h

1. Build an Evidently **Report** (`DataDriftPreset` + `DataQualityPreset`) and a **TestSuite** with explicit `gte` / `lte` conditions. Explain the split in one line: **Report is for humans, TestSuite is for machines**. Only a TestSuite can be a gate.
2. Create the **drift metrics table** in the PostgreSQL instance you already run for MLflow:

```
CREATE TABLE drift_metrics (
  id          SERIAL PRIMARY KEY,
  computed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  dataset     TEXT NOT NULL,
  feature     TEXT NOT NULL,
  metric      TEXT NOT NULL,      -- ks | chi2 | wasserstein | js
  value       DOUBLE PRECISION NOT NULL,
  threshold   DOUBLE PRECISION,
  drifted     BOOLEAN NOT NULL,
  model_version TEXT
);
```

Extract the numbers from the Evidently snapshot **programmatically** — never by parsing the HTML — and insert them.

3. Schedule the job as an Airflow DAG (`monitoring_dag.py`) running daily. This is your *scheduled* tier from Step 1; the Prometheus metrics are the *continuous* tier.
4. Add the **PostgreSQL datasource** to Grafana (provisioned from the repo, like everything else) and build the **drift history row** from SQL. Prometheus is the wrong store for a metric computed once a day and kept for a year; Postgres is the right one. **Being able to say why you used two stores is the point of this step**.
5. **Gate CI on it**: a workflow step that fails the build if the TestSuite fails on the latest batch. Screenshot the red check.
6. **Close the loop**. Add a `BranchPythonOperator` to the monitoring DAG: if drift exceeds threshold → trigger retraining (fire the `repository_dispatch` at the Module 2 CT workflow, or trigger the Module 3 training DAG directly); otherwise → skip. Add a **guard against retraining storms**: a cooldown period, a minimum sample count since the last retrain, and a hard daily cap. Justify each parameter.
7. Trigger drift end to end and capture the whole chain in screenshots:

drifted data → Evidently TestSuite fails → row written to PostgreSQL
 → Grafana drift row updates → Prometheus alert → Slack
 → Airflow branch → CT workflow → new model in Staging
 → human approves → Production → serving layer picks it up with no code change

Put that chain at the top of `reports/module-5.md`. **It is the single best artifact this course produces, and it is what you show in an interview.**

VERSION WARNING

Evidently's API changed at 0.7 — most blog tutorials you will find show deprecated imports. **Pin your Evidently version in `pyproject.toml`** and verify import paths and snapshot key paths against the docs for that exact version, not against a Medium post.

11 Logs, metrics and traces — and the schema that joins them 75 min

1. Write the **three pillars** in your report in your own words, with a concrete example from your own service of a question *only* that pillar can answer:

PILLAR	ANSWERS	YOUR EXAMPLE
Metrics	Is something wrong, right now, at what scale?	
Logs	What exactly happened in <i>this one</i> request?	
Traces	Where did the time go across the whole call chain?	

2. Define the **structured prediction-event schema** — the deliverable of this step. Every prediction emits one JSON log line carrying: `timestamp`, `correlation_id`, `model_version`, `stage_durations`, `input_summary` (hashes and summary stats, **never raw PII**), `prediction`, `confidence`, and any guardrail decisions. **Every field that is too high-cardinality to be a Prometheus label belongs here** — which is precisely the trade-off from Step 3, now resolved.
3. Prove the join works: take a latency spike on your Grafana panel, and from the timestamp and `model_version` retrieve the exact log lines for the slow requests. **A metric that you cannot drill from into individual events is a dead end at 3am.**
4. Stand up a log pipeline. Minimum viable: **Fluent Bit → Elasticsearch → Kibana** in Compose, with one saved Kibana query over your prediction events. If your machine cannot take Elasticsearch, run **Loki + Promtail** instead — it is far lighter and shares Grafana — and say in your report why you chose it.
5. Write one paragraph on **cost**: with ELK, **indexing is the cost driver, not storage**. Which of your fields do you actually need indexed, and which are retrieve-only? Then state when you would pick ELK over Loki over a managed service.

Part E — LLM observability

Required if your final project has an LLM component. If not, do Step 12 on a small toy pipeline and treat 13–15 as optional.

12 Langfuse: trace an LLM pipeline 90 min

1. Write, first, **why LLM apps fail differently** — the framing that justifies this whole half: no ground truth at request time • non-determinism, so the same input can fail once and pass twice • **the prompt is production config**, changed without a deploy • cost is per token, so a bug shows up on the invoice • and failures look like fluent, confident text rather than a stack trace. Nothing in Part B catches any of these.
2. Self-host Langfuse via Docker Compose; create a project and API keys.
3. Build or wire up a **RAG pipeline** — 20–50 documents (your final project corpus if you have it), a vector store, retrieval, and generation via your Module 3 vLLM endpoint or an API.
4. Instrument with the `@observe` decorator (or the OpenAI / LangChain integration) so one user question produces **one trace with nested spans**: `retrieve` → `rerank` → `generate`. Learn the four objects and use each: **trace** (one user interaction), **span** (one step), **generation** (a model call, with token counts), **score** (a quality judgement attached after the fact).
5. Attach **metadata to every trace**: `user_id`, `session_id`, model name, prompt version, retrieval top-k, and your `correlation_id` from Step 11 — so a trace, a log line and a metric all point at the same request.
6. Screenshot a trace waterfall and state which span owns the latency.

13 Prompt management, cost and latency 75 min

1. Move your prompt **out of the code and into Langfuse** as versioned, labelled config. Then do a promotion and a rollback **as a label move, not a deploy**, and time it. Compare that number against your Module 3 canary rollback time and write one sentence on what that changes about how you ship prompt fixes.
2. Show two traces side by side running different prompt versions, and confirm every trace records which version produced it. **A prompt change you cannot attribute to an outcome is an untracked production change.**
3. Set up **full cost tracking**, including **custom pricing for your self-hosted model** — Langfuse will not know what your own GPU costs, so derive a per-token price from your instance hourly rate and your measured throughput, and configure it. Report **cost per trace, per user and per feature**, then name your top cost driver. It is usually one of: oversized retrieved context, an unnecessarily large model on an easy path, or retry loops.
4. Track **latency properly for generation**: **time to first token**, **tokens per second**, and **p50/p95/p99 by pipeline step**. TTFT is what the user perceives as speed; total duration is not.
5. Mirror the cost and token counters into Prometheus (`llm_tokens_total{model,type,feature}`) and add a Grafana panel with daily and projected monthly spend, plus a **budget alert** to Slack — one of your six alert rules can be this one.

14 Evaluation without ground truth 2 h

1. **Write your application's scorecard first.** There is no universal LLM metric — a RAG system, an agent, a summarizer, a classifier, a chatbot, a code generator and an Arabic NLP application each need a different one. State your application type and the 3–5 metrics that define quality *for it*, and why. For Arabic specifically, note what generic English-trained metrics will get wrong about your outputs.
2. Build an evaluation set of **at least 30** examples with ground-truth contexts or reference answers.
3. Run **RAGAS** and compute the four metrics, being able to say what each one blames:
 - **Faithfulness** — grounded in the retrieved context? (*blames the generator*)
 - **Answer relevancy** — does it address the question? (*blames the generator*)
 - **Context recall** — did retrieval find everything needed? (*blames the retriever*)
 - **Context precision** — is the retrieved context mostly signal? (*blames the retriever/reranker*)
4. Build one **LLM-as-a-judge** yourself: write the rubric explicitly (criteria, a scale, and an example of each score point), run it, and **push the scores back onto the Langfuse traces** so evaluation and tracing live in one place.
5. **Calibrate the judge before you trust it.** Hand-label 20 examples yourself, run the judge on the same 20, and report the agreement rate. Then test for the three biases: **position bias** (swap the order of two candidate answers — does the verdict change?), **verbosity bias** (does the longer answer win regardless of quality?), and **self-preference** (does a judge prefer output from its own model family?). **An uncalibrated judge is a number, not a measurement.**
6. Run an **ablation**: change one thing — chunk size, top-k, or the prompt — and re-run. Show which metric moved. A metric that never moves is not measuring anything; this step proves your evaluation has signal.
7. Split evaluation into its two production roles and implement both: **offline evaluation as a CI gate** (the build fails if faithfulness drops below your threshold on the fixed test set — same pattern as the model quality gate in Module 2) and **online sampling in production** (score a small percentage of live traffic continuously, since your test set will not contain what real users do tomorrow).

15 Self-hosted LLMs, guardrails and rate limits 2 h

1. **Monitor your own vLLM deployment** (or TGI). Scrape its `/metrics` and dashboard the signals that actually predict trouble:

SIGNAL	WHAT IT TELLS YOU
Running vs waiting requests	Waiting climbing while running is flat = you are at capacity, not slow
KV cache utilization	The real capacity limit of an LLM server. Near 100% means the next request queues
Preemptions	Requests being evicted and recomputed — invisible in latency averages, brutal in p99

SIGNAL

WHAT IT TELLS YOU

TTFT and throughput

Perceived speed, and what you are paying for

Add at least one alert on this group — KV cache utilization or a preemption rate above zero.

2. Implement **input guardrails**, each one instrumented so that **every decision is a metric** (`guardrail_decisions_total{guardrail,decision}`): PII redaction · prompt injection detection · off-topic routing · retrieval ACL checks, so a user never gets context from a document they are not allowed to read.
3. Implement **output guardrails**, same instrumentation: schema validation (does the JSON parse and match?) · groundedness (does the answer follow from the retrieved context?) · toxicity · PII leakage · your own business rules.
4. For each guardrail record the **added p95 latency and cost per request**, then write the production decision: which run **inline and blocking**, and which run **asynchronously and logging-only**. Also report the **false-positive rate on clean traffic** — over-aggressive redaction breaks products, and for Arabic names and Egyptian phone or national-ID formats the off-the-shelf detectors are often wrong. Test with real Arabic inputs and say what you found.
5. Build a small **adversarial test set** for prompt injection: direct ("ignore previous instructions..."), encoded payloads, and — the one that matters for RAG — **an instruction hidden inside a retrieved document**. Report your detection rate on each category.
6. Implement **rate limiting and a budget cap** with a **Redis token bucket**, per user and per API key. Return an **honest 429 with a `Retry-After` header** — a rejection a client can act on is a feature; a hang is an outage. Then handle **upstream 429s** from your own model provider with exponential backoff and jitter, and expose both as metrics so the dashboard distinguishes "we throttled them" from "we got throttled".

16 The runbook, and release 60 min

1. Write `docs/runbook.md` — **this is a graded deliverable, not documentation garnish**. The test: **an on-call engineer who has never seen your code, woken at 3am, can follow it**. One section per alert, each with: what the alert means in one sentence · the first command to run · how to tell a real problem from a data-pipeline failure · the rollback command from Module 3 · and when to escalate rather than keep debugging.
2. Include your **symptom → cause table** from Step 9 and the **three-way GPU diagnosis** from Step 8 — those are exactly what someone needs at 3am and cannot derive under pressure.
3. Have a classmate read it cold and try to follow one section. Fix everything they had to ask about. **Their questions are the runbook's bug list**.
4. List **ten monitoring anti-patterns** you would now avoid, drawn from what you actually built.
5. Update the top-level `README.md` into a full system overview with an architecture diagram spanning all five modules, and embed the Grafana screenshot.

PULL REQUEST – MODULE 5

Branch: `module-5-observability` → **Tag after merge:** `v0.5.0`

Paste into the PR: the closed-loop screenshot chain, the four-row Grafana dashboard, the Slack alerts, your symptom → cause table, and the Langfuse trace waterfall.

Ask your reviewers to check specifically: would these alert thresholds fire on real traffic, or cause alert fatigue by week two? Does every alert annotation give an obvious first action? Can they delete the Grafana volume, `docker compose up`, and get the dashboard back? And — the real test — **can they follow your runbook cold, without asking you anything?**

OUTPUT – WHAT YOU SHIP

1. A **drift simulator** covering four drift types and four dynamics, plus a written three-tier monitoring strategy
2. **Six detection methods implemented and unit-tested** — KS, Chi-square, Wasserstein, JS/KL, MMD with a domain classifier, ADWIN/DDM — with a detection matrix and a measured false-positive rate on the control
3. A written **metric contract** with a computed cardinality budget and deliberately chosen histogram buckets
4. A real `/metrics` **endpoint** with per-stage timing, correct under multiple workers, plus `node_exporter`, `cAdvisor`, `blackbox` and (if applicable) `DCGM`, `Triton` or `vLLM` exporters
5. **PromQL you wrote yourself** for the questions that matter, plus recording rules with a measured speed-up
6. A **four-row Grafana dashboard provisioned from your repository**, with deploy annotations and template variables
7. **Six or more alert rules** with Alertmanager routing, grouping and inhibition, symptom/cause classification, and one multi-window SLO burn-rate alert
8. The **GPU/host diagnosis** — thermal throttling vs power capping vs a starving GPU — and USE-based capacity numbers
9. **Label-free model quality signals** and a symptom → cause diagnosis table for your own service
10. A **scheduled Evidently suite writing to PostgreSQL**, surfaced in Grafana via the SQL datasource, gating CI, and branching in Airflow into retraining
11. **The closed loop** — drift → alert → automated retraining → human-approved promotion → live serving — captured end to end
12. A **structured prediction-event log schema** that joins a log line back to a metric, running through a real log pipeline
13. (LLM) A **fully traced pipeline in Langfuse** with versioned prompts, label-move rollback, cost per trace/user/feature, and TTFT

14. (LLM) An **application-specific scorecard**, RAGAS results, a **calibrated** LLM judge with bias tests, and evaluation as both a CI gate and online sampling
15. (LLM) **vLLM internals monitored**, input and output guardrails instrumented as metrics with measured cost, and Redis token-bucket rate limiting with honest 429s
16. A **3am-proof runbook**, ten anti-patterns, and a complete system README

DEFINITION OF DONE

- ☐ Four drift types and four dynamics simulated; taxonomy table and three-tier strategy written
- ☐ All six detection methods implemented with tests; domain classifier AUC reported on control and drifted sets
- ☐ Detection matrix complete; **no method fires on the undrifted control**
- ☐ Metric contract written before instrumentation; cardinality budget computed; excluded labels named
- ☐ Raw exposition format annotated in the report
- ☐ Service instrumented with per-stage histograms; multiprocessing fix proven with 4 workers (before/after)
- ☐ node_exporter, cAdvisor and blackbox deployed; DCGM/Triton/vLLM where applicable; all targets UP
- ☐ `prometheus.yml` written by hand with at least one relabel config
- ☐ Five+ PromQL queries written and explained; ≥ 3 recording rules with a measured panel speed-up; retention and Thanos/Mimir paragraph written
- ☐ Four-row Grafana dashboard **provisioned from the repo** — survives deleting the container and volume
- ☐ Deploy annotations and template variables working
- ☐ ≥ 6 alert rules with `for:`, severity, routing, grouping and inhibition; each labelled symptom or cause; each annotation has an obvious first action
- ☐ One multi-window SLO burn-rate alert implemented and explained
- ☐ ≥ 3 alerts triggered deliberately and screenshotted in Slack
- ☐ Resource row built; thermal-vs-power-vs-starving diagnosis written (or the documented CPU-path equivalent)
- ☐ Per-stage timing shows which stage owns p95; queue duration vs compute time on the dashboard where applicable
- ☐ ≥ 4 label-free quality signals implemented; symptom $\rightarrow$ cause table complete with a discriminating metric per row
- ☐ USE capacity numbers backed by a Locust run

- ☐ Evidently Report + TestSuite; version pinned; results written to PostgreSQL programmatically
- ☐ Monitoring DAG scheduled in Airflow; Grafana SQL datasource provisioned; drift history row live
- ☐ CI gated on the TestSuite (red screenshot); Airflow branch triggers retraining; storm guard implemented and justified
- ☐ Closed loop demonstrated end to end with a screenshot chain
- ☐ Three pillars written with a question only each can answer; prediction-event schema defined with no raw PII
- ☐ Metric → log drill-down demonstrated on a real latency spike; log pipeline running; indexing cost paragraph written
- ☐ (LLM) Langfuse self-hosted; traces with nested spans, generations and scores; metadata including `correlation_id`
- ☐ (LLM) Prompts versioned in Langfuse; rollback performed as a label move and timed; cost per trace/user/feature with custom self-hosted pricing; TTFT and tokens/sec tracked
- ☐ (LLM) Application scorecard written; ≥ 30 eval examples; RAGAS four metrics; judge calibrated against 20 human labels with agreement rate; position, verbosity and self-preference biases tested
- ☐ (LLM) Ablation shows a metric move; evaluation runs as a CI gate **and** as online sampling
- ☐ (LLM) vLLM running/waiting, KV cache and preemptions dashboarded with an alert; input and output guardrails instrumented with measured latency, cost and false-positive rate on clean Arabic text; injection test set with per-category detection rate; Redis token bucket with `Retry-After` and upstream-429 handling
- ☐ `docs/runbook.md` written and **tested by a classmate reading it cold**; 10 anti-patterns listed
- ☐ `reports/module-5.md` complete; PR reviewed by two peers and merged; `v0.5.0` tagged

FAILURE MODES

MISTAKE	COST
Putting <code>user_id</code> or <code>request_id</code> in a Prometheus label	Cardinality explosion. You will take down your own Prometheus, and it is the classic way to do it
Ignoring the unicorn multiprocessing trap	Counters that appear to go backwards and every number above them silently wrong
Alerting on statistical significance at high traffic	Alert fatigue by week two; the team mutes the channel and the real drift is missed
Alerts with no <code>for:</code> clause	A single scrape blip pages someone at 3am

MISTAKE

COST

Alerts with no first action in the annotation	The on-call engineer stares at "drift detected" and goes back to sleep
A dashboard clicked together in the UI	Lost on the next machine, invisible to your reviewer. It was never infrastructure
Reading GPU utilization alone	You see 95% and conclude the GPU is busy. SM_ACTIVE says it is starving
Monitoring only accuracy	Ground truth arrives days late; you find out after the users do
Retraining on every drift alarm	A retraining storm on a broken upstream pipeline, and a worse model each round
Storing daily drift scores in Prometheus	Wrong tool. Long-horizon, low-frequency metrics belong in PostgreSQL
Copying Evidently code from an old blog post	Import errors from the pre-0.7 API, and an hour lost
Trusting an LLM judge you never calibrated	A confident number that measures verbosity rather than quality
Guardrails with no measured cost	400 ms of added p95 that nobody accounted for
Silently dropping requests instead of returning 429	Clients cannot back off. A rate limit becomes an outage

RESOURCES – MODULE 5

DRIFT DETECTION

- Evidently AI docs: docs.evidentlyai.com
- **Evidently's free Open-Source ML Observability Course — 40 lessons with video and code, the single best free companion to this module:** evidentlyai.com/ml-observability-course
- Course code repository: github.com/evidentlyai/ml_observability_course
- River — streaming ML, ADWIN and DDM: riverml.xyz
- SciPy statistical tests — KS, Chi-square, Wasserstein: docs.scipy.org — [stats](#)
- Alibi Detect — MMD and classifier-based drift detectors, worth reading even if you implement your own: docs.seldon.io — [alibi-detect](#)
- NannyML — performance estimation without ground truth: nannyml.readthedocs.io

PROMETHEUS

- Prometheus docs: prometheus.io/docs

- **Metric types** — read before choosing counters vs gauges: [prometheus.io — metric types](https://prometheus.io/metric-types/)
- **Metric and label naming conventions**: [prometheus.io — naming](https://prometheus.io/naming/) · **Instrumentation best practices (read the cardinality section twice)**: [prometheus.io — instrumentation](https://prometheus.io/instrumentation/)
- Histograms and summaries: [prometheus.io — histograms](https://prometheus.io/histograms/)
- PromQL basics: [prometheus.io — querying](https://prometheus.io/querying/) · Recording rules: [prometheus.io — rules](https://prometheus.io/rules/)
- Relabeling and service discovery: [prometheus.io — configuration](https://prometheus.io/configuration/)
- `prometheus-client` for Python, including multiprocess mode: prometheus.github.io/client_python
- Thanos: thanos.io · Grafana Mimir: grafana.com/docs/mimir

EXPORTERS

- `node_exporter`: github.com/prometheus/node_exporter · cAdvisor: github.com/google/cadvisor
- blackbox exporter: github.com/prometheus/blackbox_exporter · kube-state-metrics: github.com/kubernetes/kube-state-metrics
- **NVIDIA DCGM exporter** — and its metric reference, which is the source for Step 8: github.com/NVIDIA/dcgm-exporter
- `pynvml` / NVIDIA Management Library: [docs.nvidia.com — NVML](https://docs.nvidia.com/NVML/) · `psutil` for the CPU path: psutil.readthedocs.io
- Triton metrics: [docs.nvidia.com — Triton metrics](https://docs.nvidia.com/Triton/metrics/) · vLLM metrics: docs.vllm.ai

GRAFANA AND ALERTING

- Grafana docs: grafana.com/docs · **Provisioning dashboards and datasources from files**: [grafana.com — provisioning](https://grafana.com/provisioning/)
- Grafana PostgreSQL datasource: [grafana.com — postgres datasource](https://grafana.com/postgres-datasource/)
- Alertmanager configuration — routing, grouping, inhibition: [prometheus.io — alerting config](https://prometheus.io/alerting-config/)
- **Google SRE Workbook — Alerting on SLOs, the source of multi-window burn-rate alerting**: sre.google/workbook/alerting-on-slos
- Google SRE Book — *Monitoring Distributed Systems*, the four golden signals: sre.google/sre-book
- Brendan Gregg — the USE method, used in Step 9: brendangregg.com/usemethod

LOGS AND TRACES

- Elastic Stack guide: elastic.co/guide · Fluent Bit: docs.fluentbit.io · Filebeat: [elastic.co — filebeat](https://elastic.co/filebeat)
- Grafana Loki — the lighter alternative: grafana.com/docs/loki
- OpenTelemetry — the standard underneath most modern tracing: opentelemetry.io/docs

LLM OBSERVABILITY AND EVALUATION

- Langfuse docs: langfuse.com/docs · self-hosting: langfuse.com/self-hosting

- Langfuse prompt management: langfuse.com/docs/prompts • scores and evaluation: langfuse.com/docs/scores
- RAGAS: docs.ragas.io
- Zheng et al., *Judging LLM-as-a-Judge* (2023) — the paper that documents position, verbosity and self-preference bias: arxiv.org/abs/2306.05685
- Evidently's LLM evaluation course — 10 free hands-on tutorials on judges and RAG evaluation: evidentlyai.com/courses

GUARDRAILS, SECURITY AND RATE LIMITING

- Microsoft Presidio — PII detection and anonymization: microsoft.github.io/presidio
- Guardrails AI: guardrailsai.com/docs • NVIDIA NeMo Guardrails: docs.nvidia.com/nemo/guardrails
- **OWASP Top 10 for LLM Applications** — prompt injection is #1; read it once: owasp.org — LLM Top 10
- Redis rate limiting patterns for the token bucket: redis.io — rate limiting
- MDN on `Retry-After`: developer.mozilla.org — Retry-After

CERTIFICATE TRACK

The final project

The mini projects teach you the system. The final project is where you prove you can build it yourself — and it is the only thing that earns the project certificate, the rubric score, and the referral to partner companies.

YOUR FINAL PROJECT IS NOT A NEW PROJECT

It is **your model dropped into the system you have already built**. Everything the rubric asks for, you built once in the mini projects.

WHAT THE FINAL PROJECT NEEDS	ALREADY BUILT IN
Packaged, tested, containerized code with a 3-command README	Mini Project 1
MLflow tracking, registry, DVC versioning, CI/CD with a quality gate	Mini Project 2
Production serving, load testing, canary release	Mini Project 3
Optimization and the benchmark table	Mini Project 4
Monitoring stack, drift detection, LLM observability, guardrails	Mini Project 5

Two certificates

ATTENDANCE CERTIFICATE – FREE

Awarded to everyone who completes the course.

- Attend all 5 sessions (or watch the recording)
- Fill the quick feedback form after **each** session
- Fill the final course feedback form at the end
- That is it — no project required

PROJECT CERTIFICATE – OPTIONAL

Awarded for completing the full end-to-end project.

- Complete the end-to-end project — choose 1 of 2 tracks
- Submit your GitHub repo + README
- Review at least 1 other student's project
- Write detailed written feedback (min. 300 words)

HOW ATTENDANCE IS COUNTED

After each session a short feedback form is shared in the group. It takes 2 minutes. **Fill it → you are marked present. Skip it → that session does not count, even if you were in the room.**

Four rules

#	RULE	WHAT IT MEANS
1	Choose once	Deep Learning or LLM — commit at Session 1 and build the same project across all 5 sessions.
2	Commit per session	At least one GitHub commit per session. Code is your evidence of learning.
3	Same rubric	10 identical criteria. No track is easier than the other.
4	Peer review is required	You cannot earn the project certificate without reviewing another student.

Before you start

You will get far more out of the five sessions if you arrive with these two things done. Both are free — do them **before** Session 1, not during it.

PREREQUISITES FOR THE FINAL PROJECT

- 1. Study the fundamentals.** At minimum, the supervised learning course from Andrew Ng's Machine Learning Specialization on Coursera. You need to understand what a model is before you learn how to ship one. → coursera.org — Machine Learning Specialization
- 2. Train one model yourself.** At least one Kaggle project that trains a simple ML model — any data type: video, images, CSVs, audio, text. The subject does not matter. Having trained something end to end does. → kaggle.com/learn

Choose your track

Two projects, one standard. **Same 10-point rubric • same deadline • same certificate.**

Project 1 — Deep Learning • Arabic Sentiment Analysis

E-commerce product reviews in Arabic • PyTorch • AraBERT • BentoML • TensorRT • Triton

The business problem. An Arab e-commerce platform (Noon.com, Amazon.eg) receives 50,000 Arabic product reviews per day. They need to classify each as Positive, Negative or Neutral — in Arabic — to power product rankings, seller ratings and customer service prioritization.

Why this model for this course: GPU optimization and TensorRT actually matter here • Arabic NLP is directly relevant to MENA students • the drift is real (Ramadan vocabulary, slang, dialect shifts) • all three inference patterns apply • and it distills cleanly from a 12-layer teacher to a 6-layer student.

Best if you have a GPU and are interested in model optimization.

Project 2 — LLM / RAG • Arabic Legal Document Q&A

Arabic legal documents • Embeddings + Vector DB • vLLM • RAGAS • Langfuse • Guardrails

The business problem. A law firm or government authority in the Arab world needs a system that answers questions about Arabic legal documents — contracts, Egyptian Civil Code articles, Saudi regulations — accurately and with citations. Hallucinations are legally unacceptable.

Why this project for this course: RAGAS evaluation is genuinely hard • no GPU required to start, an API-based LLM works fine • embedding drift is real as legal language shifts across jurisdictions • Langfuse tracing reveals the full RAG chain • and AWQ quantization makes the LLM run fully offline.

Best if you have no GPU — the track is focused on evaluation and observability.

YOUR INPUT DATA — PROJECT 2

The corpus is the Egyptian Civil Code (القانون المدني المصري, Law of 1948) — a bilingual Arabic/English PDF, article by article.

This PDF is not your corpus. It is your raw input. Your first deliverable on this track is a script that converts it into **structured JSON**, committed to the repo and tracked with DVC. Do not embed raw `pdftotext` output — a RAG system built on unstructured page text cannot cite an article number, and an answer a lawyer cannot verify is worthless.

STEP 0 FOR PROJECT 2 — BUILD THE CORPUS

1. Target schema. One record per article. Start here and extend if your ingestion needs it:

```
{
  "article_number": 147,
  "book": "Obligations or Personal Rights",
  "chapter": "Sources of Obligations",
  "section": "Contracts",
  "topic": "The Effects of a Contract",
  "text_ar": "... العقد شريعة المتعاقدين، فلا يجوز نقضه ولا تعديله",
  "text_en": "The contract makes the law of the parties. It can be revoked ...",
  "is_repealed": false,
  "source_page": 34,
  "citation": "Egyptian Civil Code, Article 147"
}
```

2. What will actually break during extraction — budget an evening for this, and write what you hit in your report:

- **The bilingual two-column layout interleaves.** Raw extraction mixes Arabic and English mid-sentence. You need column-aware extraction or a positional heuristic, not a naive text dump.

- **Arabic is right-to-left** and extractors frequently return reversed or disconnected glyphs. Normalize with a library that handles reshaping, and **eyeball 20 random articles** before you trust the whole file.
- **Article numbers are in Arabic-Indic numerals (١٤٧)**. Normalize them to Western digits — `article_number` must be an integer you can sort, range-query and cite.
- **Some articles are repealed** (Articles 54–80 were repealed by Presidential Decree). Flag them rather than deleting them: **"that article no longer exists" is a correct and valuable answer**, and a system that silently omits them will hallucinate around the gap.
- **Headings carry the hierarchy** — Book, Chapter, Section, and the topic labels. Capture them as metadata during the pass; they become your retrieval filters and they are far cheaper than re-parsing later.
- **Arabic diacritics, hamza forms and alef variants** need normalizing consistently, or the same word will embed as two different tokens.

3. Validate before you embed. Write a test asserting: article numbers are contiguous with no unexplained gaps, every record has non-empty `text_ar`, no record exceeds a sane length (a giant record means a failed split), and repealed articles are flagged. **A silent parsing bug becomes a hallucination three steps later, and by then you will blame the model.**

4. Chunk by article, not by token count. A legal article is a self-contained unit of meaning with a natural citation. Fixed 512-token windows cut mid-provision and destroy the citation. Long articles with numbered paragraphs may split by paragraph while keeping the article number on every chunk. **Chunking strategy is exactly the MLflow experiment your Module 2 checklist asks for** — log `chunk_size`, `overlap`, `embedding_model` and RAGAS faithfulness per run, and let the numbers decide.

5. Make it reproducible. The conversion script is a `dvc.yaml` stage, and both the PDF and the resulting JSON are DVC-tracked, so `dvc repro` rebuilds the corpus from the source PDF and your reviewer can regenerate it exactly.

6. Cite by article number. Your `/ask` response returns `sources` as article citations, not chunk IDs or page numbers. That is how a lawyer verifies an answer, and it is also what makes RAGAS **context precision** and **faithfulness** meaningful for you rather than abstract.

TWO THINGS THIS CORPUS GIVES YOU FOR FREE

A bilingual evaluation set. Every article exists in Arabic and English, so you can ask the same question in both languages and check that retrieval returns the same article — a real, cheap test of whether your embedding model actually handles Arabic, and a strong ablation for your Module 5 scorecard.

And a real embedding-drift scenario: a corpus of contract law queried about family law is a genuine semantic shift, which is exactly what your MMD and domain-classifier detectors from Mini Project 5 are for.

What you ship, module by module

OUTPUT – PROJECT 1 • DEEP LEARNING

Package + API	AraBERT wrapper. <code>/predict: {"text": "رائع المنتج"} → {"label": "positive", "confidence": 0.94}</code>
Track + Version	MLflow: 5+ runs. DVC: labeled reviews. GitHub Actions CI on push.
Serve	BentoML micro-batching. TensorRT FP16. Locust: p95 before vs after.
Optimize	Distil 12→6 layers. INT8 ONNX. Benchmark: accuracy × latency × size.
Monitor	Evidently PSI on vocabulary. Prometheus + Grafana. Alert: PSI > 0.25.

OUTPUT – PROJECT 2 • LLM / RAG

Package + API	<code>src/rag.py</code> (ingest + query). <code>/ask: {"question": "ما هي شروط العقد؟"} → {"answer": "...", "sources": [...]}</code>
Track + Version	MLflow: chunking experiments (chunk_size, overlap, RAGAS faithfulness). DVC: document corpus.
Serve	BentoML wraps RAG. vLLM serves the generative model. Streaming <code>/ask</code> .
Optimize	AWQ-4bit generative model. Re-ranker distillation. RAGAS before vs after.
Monitor	RAGAS on ≥ 50 questions. Langfuse traces every request. Cosine drift. Token cost.

ORDERING NOTE

Optimization is delivered in **Week 5** and Observability in **Week 6**, so the two rows above are listed in delivery order. If an earlier version of the project guide shows Monitor before Optimize, the deliverables themselves are identical — only the week shifts.

The 10-point rubric

Same criteria for both tracks. Your project certificate and your referral eligibility are decided here.

#	AREA	MODULE	WHAT YOU BUILD	PASS CONDITION
01	Code & packaging	M1	<code>pyproject.toml</code> , <code>src/</code> layout, OOP model class, type hints	<code>pip install -e .</code> works. Model loads without errors.
02	API endpoint	M1	<code>/predict</code> with Pydantic, <code>/health</code> , async handlers	<code>curl /predict</code> returns correct output.
03	Docker	M1	Dockerfile, <code>docker-compose.yml</code> , 3-command README	<code>docker compose up</code> && <code>curl localhost:8000/predict</code> works on the reviewer's machine.
04	MLflow tracking	M2	≥ 5 runs, params + metrics + artifacts, best model in Registry	MLflow UI shows the comparison. Model promoted to Production.
05	DVC data versioning	M2	Dataset tracked, <code>dvc.yaml</code> pipeline, <code>dvc repro</code> works	<code>git checkout + dvc pull + dvc repro</code> reproduces the result.
06	GitHub Actions CI/CD	M2	Lint → test → build → push on every PR. Quality gate.	Merge a PR. Actions green. Docker image pushed to the registry.
07	Production serving	M3	BentoML (DL) or vLLM + BentoML (LLM). Locust tested.	p95 latency documented. Locust report in <code>/reports/</code> .
08	Monitoring	M5	≥ 1 drift metric (DL) or RAGAS faithfulness (LLM). Grafana panel.	Screenshot in README. Alert threshold set.
09	Peer review	Post-course	Written review ≥ 300 words, all 5 areas	Submitted on time. Covers setup, code, strength, 2 improvements, extension.
10	README & architecture	M4/M5	3-command setup, architecture diagram, session changelog	The reviewer runs it without asking you anything.

Completion checklists

PROJECT 1 · DEEP LEARNING – CODE, PACKAGING, TRACKING, VERSIONING

- ☐ `src/` layout with `pyproject.toml` — `pip install -e .` works
- ☐ AraBERT or CAMEL-BERT loaded in a class with type hints
- ☐ FastAPI `/predict : {text: str} → {label, confidence, model_version}`
- ☐ Pydantic rejects empty text — 422 returned, tested with `curl`
- ☐ `/health` returns `{status: healthy}` — used by the Docker healthcheck
- ☐ `docker build -t arabic-sentiment .` passes; `docker compose up` starts the service on port 8000

- ☐ README: exactly 3 commands to run on any machine
- ☐ MLflow tracking server running — UI accessible
- ☐ ≥ 5 runs: `model_name`, `lr`, `batch_size`, `accuracy`, `f1_macro`
- ☐ MLflow comparison screenshot in `/reports/mlflow_comparison.png`
- ☐ Best model registered as `ArabicSentiment` → promoted to Production
- ☐ Labeled dataset tracked with DVC — `dvc status` shows clean
- ☐ `dvc repro` reproduces training with the same metrics (± 0.005)
- ☐ GitHub Actions: lint → test → docker build → push on `main`
- ☐ CI fails if `f1_macro` drops below baseline — documented in README

PROJECT 1 • DEEP LEARNING – SERVING, MONITORING, OPTIMIZATION

- ☐ BentoML service with `batchable=True` Runner — `bentoml models list` shows the model
- ☐ `bentoml serve` runs `/predict` at `localhost:8000`
- ☐ Locust report before optimization in `/reports/locust_fastapi.html`
- ☐ TensorRT FP16 engine built — committed or documented
- ☐ Locust after TensorRT in `/reports/locust_trt.html` — p95 in README
- ☐ Batch scoring: $\geq 1,000$ samples, output to `/data/scoring/output/`
- ☐ nginx canary config: weight 5/95 — rollout stages in README
- ☐ Evidently DataDrift report on the latest batch — saved in `/reports/`
- ☐ PSI computed for `text_length` and `confidence_score` features
- ☐ Drift check triggered after every batch scoring run
- ☐ Prometheus exposes the PSI score at `/metrics`
- ☐ Grafana: ≥ 2 panels (p95 + PSI) — screenshot in README
- ☐ Alert: $\text{PSI} > 0.25$ → notification documented in README
- ☐ 12-layer AraBERT distilled to a 6-layer student — model file committed
- ☐ INT8 ONNX quantization applied — `model_int8.onnx` committed
- ☐ Benchmark table: original vs distilled vs INT8 (accuracy, p95, size)
- ☐ Final architecture diagram in README covering all 5 sessions

PROJECT 2 • LLM / RAG – CODE, PACKAGING, TRACKING, VERSIONING

- ☐ `src/` layout with `pyproject.toml` — `pip install -e .` works

- ☐ Egyptian Civil Code PDF converted to **structured JSON**, one record per article, committed to the repo
- ☐ Extraction validated: article numbers normalized to integers, Arabic text spot-checked on 20 articles, repealed articles flagged
- ☐ Ingestion pipeline: JSON → chunk by article → embed → vector store
- ☐ FastAPI `/ask : {question: str} → {answer: str, sources: list[str]}` — **sources are article citations, not chunk IDs**
- ☐ Pydantic rejects an empty question — 422 returned and tested
- ☐ `/health` returns `{status: healthy, documents_indexed: N}`
- ☐ Dockerfile includes the vector store and embedded documents; `docker compose up` starts on port 8000
- ☐ README: 3 commands to run Q&A on any machine
- ☐ MLflow logs each config: `chunk_size`, `overlap`, `embedding_model`, `faithfulness`
- ☐ ≥ 5 runs compared — MLflow screenshot in `/reports/`
- ☐ Best chunking config registered in the MLflow Registry
- ☐ Source PDF and derived JSON both tracked with DVC — `dvc pull` fetches them, `dvc repro` rebuilds the JSON from the PDF
- ☐ `dvc repro` re-indexes the corpus reproducibly from tracked documents
- ☐ GitHub Actions: lint → test → rebuild index → push Docker image
- ☐ CI fails if RAGAS faithfulness < 0.75 on a 20-question test set

PROJECT 2 · LLM / RAG – SERVING, MONITORING, OPTIMIZATION

- ☐ BentoML wraps the RAG pipeline with an async `/ask` endpoint
- ☐ vLLM serves the generative LLM — model name in README
- ☐ Streaming: tokens appear progressively in `curl` output
- ☐ Locust report at 50 concurrent users in `/reports/`
- ☐ Batch re-indexing script tested with at least one new document
- ☐ Canary rollout config documented in README
- ☐ RAGAS on ≥ 50 questions — all 4 metrics logged
- ☐ RAGAS results stored in MLflow — trend visible across sessions
- ☐ Alert: faithfulness < 0.80 → notification triggered
- ☐ Langfuse self-hosted — every `/ask` creates a trace with spans
- ☐ RAGAS faithfulness score attached to each Langfuse trace

Project 3 — Computer Vision • Pet Breed Classification

User-submitted pet photos • PyTorch • ResNet / MobileNet • BentoML • TensorRT • Triton • ONNX Runtime • OpenVINO

THREE TRACKS, NOT TWO

This project was added after the first print of the handbook. If an earlier copy of the course guide says “choose 1 of 2 tracks”, “Deep Learning or LLM” or “two projects, one standard”, read those as **three**. Nothing else changes: the same 10-point rubric, the same deadline, the same certificate, the same referral route. The rubric needed no new rows for this track — Track C is a deep-learning project, so rows 07 and 08 apply to it exactly as written.

The business problem. A pet-care marketplace operating across Egypt, Saudi Arabia and the UAE lets users upload a photo of their animal when they create a profile. That one photo drives everything downstream: which food and accessory listings are recommended, which veterinary content is surfaced, and which adoption listings get matched to which household. Users type the breed wrong, in Arabic, in English, or not at all. The platform needs a service that takes a phone photo and returns a breed with a calibrated confidence — and that says “*I am not sure*” instead of guessing, because a confident wrong breed sends a large-breed food recommendation to a kitten owner.

Why this project for this course. This is the track with the smallest machine-learning surface and the largest engineering surface. Twenty lines of torchvision transfer learning gets you a working model in an afternoon, which means every remaining hour of the six weeks lands on the system. It is also the only track where the *entire* Module 4 toolbox applies to one model and produces numbers you can actually compare: pruning, post-training quantization, quantization-aware training, distillation, TensorRT on GPU, ONNX Runtime and OpenVINO on CPU, and TFLite at the edge. Image drift is real and measurable here — phone cameras, indoor lighting, motion blur, and the seasonal shift when everyone photographs their pets outdoors in winter. And a fine-grained CNN is the one model in this course where a distilled student and an INT8 engine give visibly different accuracy-latency trade-offs rather than a rounding error.

Best if you are newer to deep learning and want the full optimization story. No Arabic NLP, no tokenizers, no RAG evaluation theory — a pretrained backbone, a replaced classification head, and then six weeks of production engineering. A GPU helps for Session 4; Colab’s free T4 is acceptable and is what most students on this track use.

YOUR INPUT DATA – PROJECT 3

The dataset is Oxford-IIIT Pet — 7,349 images, 37 cat and dog breeds, roughly 200 images per class, about 800 MB. It ships with `torchvision.datasets.OxfordIIITPet(download=True)`, so you are training within ten minutes of starting.

Why 37 breeds and not cats-vs-dogs. Binary cats-vs-dogs reaches 99% accuracy with a frozen backbone and one epoch. That sounds like a gift and it is a trap: with no headroom, distillation shows no loss, quantization shows no loss, pruning shows no loss, and your entire Module 4 benchmark table is five identical numbers. You will have built the pipeline and measured nothing. Fine-grained breed classification sits around 88–93% with a ResNet-50, which leaves exactly enough room for every optimization in the course to move the number in a direction you have to explain.

This dataset is not your production data. It is studio-quality, centred, well-lit, and balanced. Real uploads are none of those things. Your first deliverable on this track is a script that turns it into a realistic, versioned, drifting dataset — committed to the repo and tracked with DVC.

Approved alternatives if you prefer another domain, cleared with the instructor in Session 1: Stanford Dogs (120 classes, harder), PlantVillage crop-disease leaves (agriculture, MENA-relevant), or a recyclable-waste classifier. The rules below apply unchanged. **You may not use CIFAR-10 or the NYC TLC dataset** — those are mini-project material, and your reviewer needs to see what you built rather than what the mini projects handed you.

STEP 0 FOR PROJECT 3 – BUILD THE DATASET AND THE CORRUPTION SUITE

1. Fix the split before anything else. Oxford-IIIT Pet ships a `trainval` / `test` split. Use it, then carve a validation fold out of `trainval` with a fixed seed and commit the index files. Never touch `test` until Session 4. A test set you have looked at forty times is a validation set with a misleading name, and every number in your report inherits the lie.

2. Build a corruption suite — this is your drift ground truth. Write `data/corruptions.py` applying, at three severity levels each: Gaussian blur (out-of-focus phone), brightness shift up and down (indoor evening, direct sun), JPEG compression at quality 30 (messaging-app re-encode), downscale-then-upscale to 96×96 (old handset), and motion blur. Generate a corrupted copy of the test set per severity. You now own a labelled drift scenario: you know exactly which shift you applied and when, so in Module 5 you can score your detectors against the truth instead of guessing.

3. Target manifest schema. One record per image, committed as JSON and DVC-tracked:

```
{
  "image_id": "Abyssinian_100",
  "path": "data/images/Abyssinian_100.jpg",
  "breed": "Abyssinian",
  "species": "cat",
  "class_index": 0,
  "split": "train",
  "corruption": null,
  "severity": 0,
  "width": 394,
  "height": 500
}
```

4. What will actually break — budget an evening, and write what you hit in your report:

- **Train/serve preprocessing skew.** Your training transform resizes, centre-crops and normalizes with ImageNet mean and std. If you reimplement that by hand in the API and get the channel order or the normalization constants wrong, the model still returns a confident answer — a wrong one, with no error anywhere. Export the exact eval transform alongside the weights and load it from one place. Then write the test that proves it: the same image through the training path and through the API path must produce logits agreeing to 1e-4.
- **Class imbalance is mild here, ordering is not.** Class indices must come from a committed, sorted label map — never from directory-listing order, which differs between your laptop and the CI runner. A shuffled label map is the bug that makes your API return “Bengal” for every Beagle and passes every test you wrote.
- **Four of the 37 breeds are genuinely confusable** even for humans (the three spaniels; British Shorthair against Russian Blue). Do not chase them with more epochs. Put the confusion matrix in your report and use it to justify your abstention threshold instead — that is the production answer.
- **Confidence out of a softmax is not calibrated.** A model that is 96% confident is right well below 96% of the time, and your “*I am not sure*” behaviour depends on that number meaning something. Apply temperature scaling on the validation fold, plot the reliability diagram before and after, and set your abstention threshold on the calibrated score. Log the temperature as an MLflow parameter and ship it in the artifact.
- **Some images are CMYK, some are greyscale, one is a PNG with an alpha channel.**
`Image.open(...).convert("RGB")` at both training and serving, and a test that feeds a 4-channel PNG to `/predict` and expects 200, not 500.
- **Augmentation belongs to training only.** Random crops and flips in the eval or serving path produce different answers for the same image on two consecutive calls, and you will spend a day blaming the GPU.

5. Validate before you train. Write tests asserting: every manifest path exists and opens, no `image_id` appears in two splits, the label map has exactly 37 entries and matches the committed reference, every class has at least 50 training images, and no image is under 32×32. A silent manifest bug becomes a wrong prediction three steps later, and by then you will blame the model.

6. Make it reproducible. The manifest builder and the corruption generator are `dvc.yaml` stages. Images, manifest and corrupted sets are all DVC-tracked, so `dvc repro` rebuilds everything from the source archive and your reviewer regenerates your exact splits.

7. Abstain, do not guess. `/predict` returns the top-3 breeds with calibrated probabilities and a `decision` field that is "uncertain" when the top calibrated score falls below your threshold. Report coverage and selective accuracy — what fraction you answer, and how accurate you are on that fraction — not just top-1. That pair of numbers is what a product owner actually buys.

TWO THINGS THIS DATASET GIVES YOU FOR FREE

A drift scenario with a known answer. You generated the corruptions in Step 0, so when your Module 5 detectors fire you can check them against ground truth: which detector caught which corruption, at which severity, with how many false alarms on the clean batches. Nobody on Tracks A or B can do that — their drift is real but unlabelled. A detector scorecard in your report is one of the strongest things a reviewer can read.

The complete optimization matrix on one model. A 25M-parameter ResNet-50 and a 2.5M-parameter MobileNetV3-Small solving the same task, at the same input size, on data you already have. Every technique in Module 4 — pruning, PTQ, QAT, distillation, TensorRT, ONNX Runtime, OpenVINO, TFLite — applies to this pair without any adaptation, and each one moves accuracy and latency by an amount you can defend. No other track in this course gets a clean sweep of all eight.

OUTPUT – PROJECT 3 • COMPUTER VISION

Package + API	PyTorch model class wrapping backbone + exported eval transform. <code>/predict</code> : multipart image → <code>{"breed": "Bengal", "species": "cat", "confidence": 0.91, "top_3": [...], "decision": "confident", "model_version": "v3"}</code>
Track + Version	MLflow: ≥ 5 runs across ≥ 3 backbones (ResNet-50, ResNet-18, MobileNetV3-Small) with top-1, macro-F1, ECE, coverage and selective accuracy. DVC: images, manifest, corrupted sets. GitHub Actions CI with a top-1 quality gate.
Serve	BentoML with a batchable Runner. TensorRT FP16 engine on Triton (GPU). ONNX Runtime + OpenVINO path (CPU, mandatory). Locust: p95 before vs after.
Optimize	Five techniques benchmarked: structured pruning, INT8 PTQ, QAT, ResNet-50 → MobileNetV3 distillation, TensorRT FP16. One table: top-1 × p95 × size × hardware.
Monitor	Embedding drift (MMD or a domain classifier on penultimate-layer features) plus a cheap pixel-statistics detector. Confidence-distribution drift. Evidently report per batch. Prometheus + Grafana. Alert wired to the retraining DAG.

WHERE TRACK C IS DELIBERATELY HARDER

The model is easier to train than AraBERT, so four requirements are stricter than on Tracks A and B. This is what keeps “no track is easier than another” honest — read it before you choose this track thinking it is the light one.

1. **The full optimization matrix, not a subset.** Track A benchmarks two techniques. You benchmark five, in one table, with a `hardware` column, and you explain every row that moved the wrong way.
2. **Calibration is mandatory.** Temperature scaling, a reliability diagram before and after, and an abstention threshold defended with coverage and selective accuracy. Raw softmax confidence in your API loses points on rubric row 02.
3. **Two drift detectors, scored against ground truth.** One embedding-based, one cheap statistical, plus the detector scorecard across your corruption severities.
4. **The retraining loop must actually close and be demonstrated.** Drift alert → Airflow DAG → fine-tune on the corrupted distribution → quality gate against the Production model → promote or reject. Show the triggered run in your README.

PROJECT 3 · COMPUTER VISION – CODE, PACKAGING, TRACKING, VERSIONING

- ☐ `src/` layout with `pyproject.toml` — `pip install -e .` works
- ☐ Manifest JSON and corruption suite generated by committed scripts; corruptions and severities documented in `data/README.md`
- ☐ Label map committed and sorted; a test asserts it has 37 entries and matches the reference
- ☐ Model loaded in a class with type hints — weights and eval transform loaded once at startup, not per request
- ☐ Eval transform exported with the weights; a test asserts training-path and API-path logits agree to $1e-4$
- ☐ FastAPI `/predict`: multipart image → {breed, species, confidence, top_3, decision, model_version}
- ☐ Pydantic and file validation reject a non-image, an oversized upload and a 4-channel PNG correctly — 422 or 200, never 500, tested with curl
- ☐ Temperature scaling fitted on validation; reliability diagram before and after in `/reports/calibration.png`
- ☐ Abstention threshold set from calibrated scores; coverage and selective accuracy in README
- ☐ `/health` returns {status: healthy, model_version, num_classes} — used by the Docker healthcheck
- ☐ `docker build -t pet-breed .` passes; `docker compose up` starts the service on port 8000
- ☐ README: exactly 3 commands to run on any machine
- ☐ MLflow tracking server running — UI accessible
- ☐ ≥ 5 runs across ≥ 3 backbones: `backbone`, `lr`, `batch_size`, `top1`, `f1_macro`, `ece`, `temperature`
- ☐ MLflow comparison screenshot in `/reports/mlflow_comparison.png`
- ☐ Best model registered as `PetBreedClassifier` → promoted to Production
- ☐ Images, manifest and corrupted sets tracked with DVC — `dvc status` clean
- ☐ `dvc repro` reproduces training with the same metrics (± 0.005 top-1)
- ☐ GitHub Actions: lint → test → docker build → push on main
- ☐ CI fails if top-1 drops below baseline — documented in README

PROJECT 3 · COMPUTER VISION – SERVING, MONITORING, OPTIMIZATION

- ☐ BentoML service with `batchable=True` Runner — `bentoml models list` shows the model
- ☐ `bentoml serve` runs `/predict` at `localhost:8000`
- ☐ Locust report before optimization in `/reports/locust_fastapi.html`
- ☐ ONNX export with a test asserting PyTorch and ONNX logits agree to $1e-4$ on 200 images
- ☐ ONNX Runtime and OpenVINO CPU path live — p95 for both in README
- ☐ TensorRT FP16 engine built and served on Triton with dynamic batching — Colab T4 acceptable, hardware documented
- ☐ Locust after TensorRT in `/reports/locust_trt.html` — p95 in README
- ☐ Batch scoring: $\geq 1,000$ images, output to `/data/scoring/output/`
- ☐ nginx canary config: weight 5/95 — rollout stages in README
- ☐ Embedding drift: MMD or a domain classifier on penultimate-layer features — score logged per batch
- ☐ Cheap statistical detector on pixel brightness, contrast and sharpness — logged per batch
- ☐ Confidence-distribution drift tracked separately from input drift
- ☐ Detector scorecard: which detector caught which corruption, at which severity, with false alarms on clean batches
- ☐ Evidently report on the latest batch — saved in `/reports/`
- ☐ Drift check triggered after every batch scoring run
- ☐ Prometheus exposes drift scores and p95 at `/metrics`
- ☐ Grafana: ≥ 3 panels (p95, drift score, confidence distribution) — screenshot in README
- ☐ Alert: drift above threshold → Airflow retraining DAG triggered — triggered run shown in README
- ☐ Quality gate on retraining: promoted only if $\text{top-1} \geq \text{Production} - 0.01$
- ☐ Structured channel pruning applied — sparsity level and accuracy delta documented
- ☐ INT8 post-training quantization with a calibration set — `model_int8.onnx` committed
- ☐ Quantization-aware training run — accuracy compared against PTQ
- ☐ ResNet-50 distilled into MobileNetV3-Small — student weights committed
- ☐ Benchmark table: baseline vs pruned vs PTQ vs QAT vs distilled vs TensorRT FP16 (top-1, p95, size, hardware)
- ☐ Every row of the table that got worse is explained in one sentence — see the honesty clause
- ☐ Final architecture diagram in README covering all 5 sessions

FAILURE MODES – WHAT COSTS TRACK C STUDENTS THE MOST HOURS

- **Chasing accuracy instead of building the system.** A backbone that is 1.5% better moves zero rubric points. Freeze your model by the end of Week 2 and spend the rest of the course on the eighteen boxes above that have nothing to do with training.
- **Reimplementing preprocessing in the API.** The single most common way this track fails silently. Export the transform, load it from one place, and write the agreement test.
- **Benchmarking a Colab T4 number against a laptop CPU number in the same column.** The `hardware` column exists for a reason, and a reviewer will spot it.

- **Quantizing before you have a working benchmark harness.** An unmeasured optimization is a rumour. Module 4 says this and this track is where it bites hardest, because five techniques with no harness is five unverifiable claims.
- **Assuming “no Arabic, no RAG” means less work.** Read the four stricter requirements again. The hours move from the model to the engineering; they do not disappear.

A NOTE ON THE HONESTY CLAUSE

It applies here with extra force, because five optimization techniques give you five chances to report something that did not happen. “*INT8 PTQ cost 2.1% top-1 and QAT recovered 1.6% of it for four extra GPU hours, so I shipped FP16 TensorRT*” is a better report than a table where every technique is free. Negative results, measured properly and reported clearly, are production engineering. Fabricated numbers are the one thing that fails a submission outright.

- ☐ Cosine similarity on query embeddings vs baseline — drift logged
- ☐ Token usage in Prometheus — Grafana shows cost/hour
- ☐ Guardrails PII detection active on all `/ask` responses
- ☐ Generative model quantized to AWQ-4bit — weights documented
- ☐ RAGAS scores: original vs quantized — ≤ 0.03 faithfulness drop
- ☐ Latency before vs after quantization in README
- ☐ Final architecture diagram covering all 5 session components

Peer review

Required for the project certificate — you cannot earn it without reviewing another student. **Pairing is assigned by the instructor after Session 5.** Every review is read; quality matters.

AS THE REVIEWER – WHAT YOU SUBMIT

- Cloned the assigned student's repo and followed the README
- Service started — documented any issues encountered
- Written review ≥ 300 words as a **GitHub Issue on their repo**
- A PDF copy of the review sent to the instructor
- **Section 1 — Setup:** did it run? how long? what broke?
- **Section 2 — Code quality:** two specific patterns, one good and one to refactor
- **Section 3 — One genuine strength:** specific, in production context
- **Section 4 — Two specific improvements:** concrete and actionable
- **Section 5 — Extension idea:** what, why, how — one paragraph

AS THE REVIEWEE – WHAT YOU RESPOND

- Read the review carefully before responding
- One GitHub comment per review section
- Each improvement: implement it, or explain why you disagree
- Extension idea: acknowledge it and note whether you plan to implement it
- Response submitted within 3 days of receiving the review

How to write a good review

#	AREA	WHAT TO LOOK AT	WHAT TO WRITE
1	Can I run it?	Clone. Follow README. Does it work? How long? What broke?	Document every friction point. If it ran fine, note what was clear.
2	Code quality	Type hints? OOP? Model loaded once? Repeated code?	Name 2 patterns: one that is good, one to refactor. Be specific.
3	One genuine strength	The ONE thing done notably well. "Good code" is not useful.	One paragraph: what impressed you and why it matters in production.
4	Two improvements	Not "add more tests". Specific enough to act on immediately.	e.g. "Locust <code>wait_time</code> should be <code>between(0.5, 2.0)</code> for realism".
5	Extension idea	One concrete thing to build next to extend scope or production-readiness.	What it is, why it matters, how you would approach it — one paragraph.

YOU HAVE ALREADY BEEN PRACTISING THIS

The collaboration protocol asks you to review two pull requests per module. Ten reviews of real code, over six weeks, is why your final peer review will be worth reading. Students who review from Week 1 consistently write the strongest final reviews.

Timeline and deadlines

CHECKPOINT	WHAT MUST EXIST
End of Session 1	GitHub repo created. <code>/predict</code> running in Docker. 3-command README.
End of Session 2	≥ 5 MLflow runs logged. DVC pipeline clean. GitHub Actions green.
End of Session 3	BentoML or vLLM service running. Locust report submitted.
End of Session 4	Optimization complete. Benchmark table.
End of Session 5	Drift detection or RAGAS running. Grafana dashboard live. Start final polish.
2 weeks after Session 5	Submit your GitHub repo AND your peer review together. No separate deadlines.

Week 3 has no lecture. It is build time: apply Modules 1 and 2 to your chosen track and revise. Use it — students who treat Week 3 as a break do not finish.

Referral to partner companies

Top projects are referred to our partner companies by name — the same referral route already offered to community moderators. Referrals go out after Session 5, once peer reviews are in.

HOW YOU QUALIFY

- Complete the end-to-end project and earn the project certificate
- Score highly on the 10-point rubric — same criteria for both tracks
- Ship a repository a reviewer can run without asking a question
- Complete your peer review on time

WHAT YOU GET

Your project put in front of hiring partners, by name.

The attendance certificate alone does not qualify.

THE HONESTY CLAUSE

A report that says "I tried TensorRT INT8 and accuracy dropped 4%, so I shipped FP16" scores higher than one claiming a flawless 10× speedup with no accuracy column. **Negative results, measured properly and reported clearly, are production engineering.** Fabricated numbers are the one thing that fails a submission outright.

RESOURCES – FINAL PROJECT

- **Recorded project explanation session** — watch this before you choose your track: youtube.com/watch?v=n0dv6KZHq9M
- **Project 2 input data — the Egyptian Civil Code (bilingual Arabic/English PDF):** sadanykhalifa.com — القانون المدني المصري · convert it to structured JSON first
- **Course Guide deck** — strategy, tracks, checklists, rubric, timeline (shared on the course platform)
- Andrew Ng, Machine Learning Specialization — Course 1 minimum, before Session 1: coursera.org
- Kaggle Learn — Python + Pandas micro-courses, about 2 hours total: kaggle.com/learn
- AraBERT (Track A): github.com/aub-mind/arabert · CAMEL Tools: github.com/CAMEL-Lab/camel_tools
- How to write a good GitHub README: docs.github.com — about READMEs

Master resource index

FOUNDATIONS – DO THESE IF YOU ARE MISSING BACKGROUND

TOPIC	RESOURCE
Git	youtube.com/watch?v=N7qWNQxFq90
Linux	youtube.com/watch?v=6WatcfENsOU
Docker (MLOps MENA on Zomra)	zomra.io — free session
Airflow	youtu.be/vEApEfa8HXk
Python + Pandas (2 hours total)	kaggle.com/learn
ML fundamentals (required if you have never trained a model)	Andrew Ng — ML Specialization, Course 1 minimum
SQL	sqlbolt.com
Pull requests and code review	docs.github.com/pull-requests

FULL COURSES WORTH DOING ALONGSIDE THIS ONE

COURSE	WHY
MLOps Zoomcamp (DataTalks.Club)	Free, self-paced, project-based. Overlaps Modules 2, 3 and 5 almost exactly, and uses the same NYC TLC dataset
Made With ML (Goku Mohandas)	The strongest free material on packaging, testing and code quality for ML
Evidently ML Observability Course	40 free lessons; the deepest free treatment of Module 5's first half
Full Stack Deep Learning	Best free lectures on the <i>system design</i> around ML
LLM Zoomcamp (DataTalks.Club)	If you are on final project Track B

BOOKS

- Chip Huyen, *Designing Machine Learning Systems* — the single best book for this course:
huyenchip.com/books

- Chip Huyen, *AI Engineering* — the LLM-era companion
- Google, *Site Reliability Engineering* — free online, and the origin of the monitoring ideas in Module 5: sre.google/books
- Kleppmann, *Designing Data-Intensive Applications* — for the data layer under all of this

PAPERS WORTH ONE READ EACH

- Sculley et al., *Hidden Technical Debt in Machine Learning Systems* (2015) — [NeurIPS](#)
- Hinton et al., *Distilling the Knowledge in a Neural Network* (2015) — arxiv.org/abs/1503.02531
- Dettmers et al., *LLM.int8()* (2022) — arxiv.org/abs/2208.07339
- Lin et al., *AWQ* (2023) — arxiv.org/abs/2306.00978
- Kwon et al., *PagedAttention* — the vLLM paper (2023) — arxiv.org/abs/2309.06180
- Breck et al., *The ML Test Score* (Google, 2017) — a 28-point production readiness checklist worth auditing your repo against

COMMUNITIES

- **MLOps MENA Community** — sessions, mentorship, and the Arabic-language MLOps content this course belongs to
- MLOps.community (Slack + podcast): mlops.community
- DataTalks.Club Slack: datatalks.club/slack

Instructor slot — Arabic-language links

Every link above is in English. Paste 1–2 Arabic-language videos or community session recordings per module so students who prefer Arabic have a first-language entry point to each topic.

MODULE	TOPIC GAP TO FILL	LINK
1	FastAPI / pytest in Arabic	
2	MLflow / DVC in Arabic	
3	Serving & orchestration in Arabic	
4	Quantization & TensorRT in Arabic	

MODULE	TOPIC GAP TO FILL	LINK
5	Monitoring & LLM observability in Arabic	

The MLOps Practitioner — Mini Projects & Final Project Handbook · ITI × MLOps MENA Community · Instructor: Aya Nasser Salama

One repository · five mini projects · five pull requests · one final project. Share your repo with your study group and review each other's work — that is the course.